

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**ĐINH QUỐC CƯỜNG – 52300012
NGUYỄN THANH TÚ – 52300076
VÕ THÁI TUẤN – 52300081**

DỊCH VỤ Y TẾ SỐ SMART- HEALTHCARE APPLICATION

BÁO CÁO CUỐI KỲ MẪU THIẾT KẾ

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**ĐINH QUỐC CƯỜNG – 52300012
NGUYỄN THANH TÚ – 52300076
VÕ THÁI TUẤN – 52300081**

DỊCH VỤ Y TẾ SỐ SMART- HEALTHCARE APPLICATION

BÁO CÁO CUỐI KỲ MẪU THIẾT KẾ

Người hướng dẫn
ThS. NCS. Vũ Đình Hồng

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin được bày tỏ lòng biết ơn sâu sắc nhất đến Ban Giám hiệu nhà trường cùng các thầy cô giáo khoa Công nghệ thông tin, những người đã tận tình truyền đạt kho tàng kiến thức quý báu và tạo điều kiện học tập tốt nhất cho em trong suốt những năm vừa qua. Chính nền tảng kiến thức vững chắc và sự dạy bảo tâm huyết của các thầy cô đã giúp em có đủ tự tin và kỹ năng để thực hiện đề tài xây dựng ứng dụng chat real-time này một cách hoàn thiện nhất.

Đặc biệt, em xin gửi lời cảm ơn chân thành nhất tới thầy Vũ Đình Hồng, người đã trực tiếp hướng dẫn, chỉ bảo và đồng hành cùng em trong suốt quá trình triển khai đề án. Những lời góp ý chuyên môn sắc sảo, sự định hướng đúng đắn về mặt kỹ thuật cũng như sự động viên tinh thần của thầy/cô là yếu tố then chốt giúp em vượt qua những khó khăn trong việc xử lý các tính năng phức tạp như truyền tải video và tối ưu hóa dữ liệu thời gian thực. Nếu không có sự hỗ trợ tận tâm đó, em khó lòng có thể hoàn thành đề tài đúng tiến độ và đạt được các mục tiêu đã đề ra.

TP. Hồ Chí Minh, ngày 10 tháng 04 năm 2026.

Tác giả 1



Đinh Quốc Cường

Tác giả 2



Nguyễn Thanh Tú

Tác giả 3



Võ Thái Tuấn

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của TS. Vũ Đình Hồng. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 10 tháng 04 năm 2026

Tác giả 1



Đinh Quốc Cường

Tác giả 2



Nguyễn Thanh Tú

Tác giả 3



Võ Thái Tuấn

DỊCH VỤ Y TẾ SỐ SMART-HEALTHCARE APPLICATION

TÓM TẮT

Hệ thống ứng dụng dịch vụ y tế số Smart Healthcare là giải pháp công nghệ toàn diện nhằm tối ưu hóa quy trình chăm sóc sức khỏe thông qua việc tích hợp các công nghệ tiên tiến. Hệ thống cho phép người dùng chủ động quản lý hồ sơ sức khỏe điện tử, đặt lịch khám, và thực hiện tư vấn y tế từ xa (Telemedicine), đồng thời hỗ trợ các cơ sở y tế giảm tải vận hành và nâng cao độ chính xác trong chẩn đoán. Bằng cách kết nối dữ liệu thời gian thực từ các thiết bị đeo thông minh và áp dụng các thuật toán phân tích dữ liệu lớn, dự án không chỉ xóa bỏ rào cản địa lý mà còn hướng tới mục tiêu cá nhân hóa lộ trình điều trị, góp phần xây dựng một hệ sinh thái y tế thông minh, minh bạch và hiệu quả cho cộng đồng.

MỤC LỤC

DANH MỤC HÌNH VẼ	viii
CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI	1
1.1 Giới thiệu đề tài	1
1.2 Lý do chọn đề tài	1
1.3 Mục tiêu thực hiện đề tài	2
1.4 Phạm vi đề tài	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	4
2.1 Kiến trúc hướng dịch vụ	4
2.2 Mẫu thiết kế	5
2.3 Công nghệ sử dụng	5
CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG	6
3.1 Phân tích kiến trúc hệ thống	6
3.2 Phân tích vai trò người dùng	7
3.2.1 Bệnh nhân	7
3.2.2 Bác sĩ	7
3.2.3 Quản trị viên hệ thống	7
CHƯƠNG 4. YÊU CẦU HỆ THỐNG	8
4.1 Yêu cầu chức năng	8
4.1.1 Xác thực người dùng	8
4.1.2 Đặt lịch khám bệnh	11

4.1.3 Cập nhật trạng thái lịch khám bệnh.....	12
4.1.4 Xem hồ sơ sức khỏe.....	13
4.1.5 Xem danh sách lịch khám bệnh.....	13
4.1.6 Thông báo.....	14
4.1.7 Thay đổi thông tin cá nhân.....	16
4.2 Yêu cầu phi chức năng.....	16
4.2.1 Bảo mật và Riêng tư.....	16
4.2.2 Độ tin cậy & Sẵn sàng.....	17
4.2.3 Hiệu năng.....	17
CHƯƠNG 5. PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU.....	18
5.1 Phân tích cơ sở dữ liệu.....	18
5.2 Thiết kế dữ liệu cho các service.....	18
5.2.1 User service.....	18
5.2.2 Patient service.....	19
5.2.3 Doctor service.....	21
5.2.4 Hospital service.....	23
5.2.5 EHR service.....	24
5.2.6 Appointment service.....	26
CHƯƠNG 6. SƠ ĐỒ THIẾT KẾ CHỨC NĂNG.....	29
6.1 Sơ đồ Use-case.....	29
6.1.1 Đăng ký.....	29
6.1.2 Đăng nhập.....	31
6.1.3 Quên mật khẩu.....	32

6.1.4 Thay đổi mật khẩu.....	33
6.1.5 Đăng xuất.....	34
6.1.6 Xem hồ sơ sức khỏe.....	34
6.1.7 Xem danh sách lịch khám bệnh.....	35
6.1.8 Cập nhật thông tin cá nhân.....	36
6.1.9 Đặt lịch khám bệnh	37
6.1.10 Cập nhật trạng thái lịch hẹn	38
6.1.11 Gửi thông báo tới người dùng.....	38
6.2 Sơ đồ tuần tự	40
6.2.1 Đăng ký	40
6.2.2 Đăng nhập.....	41
6.2.3 Quên mật khẩu	42
6.2.4 Thay đổi mật khẩu.....	43
6.2.5 Đăng xuất.....	44
6.2.6 Xem hồ sơ sức khỏe.....	44
6.2.7 Xem danh sách lịch khám bệnh.....	45
6.2.8 Cập nhật thông tin cá nhân.....	46
6.2.9 Đặt lịch khám bệnh	47
6.2.10 Cập nhật trạng thái lịch hẹn	48
6.2.11 Gửi thông báo tới người dùng.....	49
CHƯƠNG 7. ÁP DỤNG PATTERN	50
7.1 Singleton Pattern	50
7.2 Strategy Pattern	50

7.3 Template Method Pattern.....	51
7.4 Simple Factory Pattern.....	52
7.5 Command Pattern.....	54
7.6 Adapter Pattern.....	55
7.7 Observer Pattern.....	57
7.8 Decorator Pattern.....	58
7.9 State Pattern.....	59
7.10 Facade Pattern	60
7.11 Lock/Guard Pattern	60
7.12 Global Exception Handler Pattern.....	61
CHƯƠNG 8. THIẾT KẾ GIAO DIỆN.....	63
8.1 Giao diện cho bệnh nhân.....	64
8.2 Giao diện cho bác sĩ	67
CHƯƠNG 9. KẾT LUẬN.....	69
9.1 Kết luận	69
9.2 Hướng phát triển.....	69
9.2.1 Nâng cấp và mở rộng Mẫu thiết kế.....	70
9.2.2 Tích hợp Trí tuệ Nhân tạo và Machine Learning.....	70
TÀI LIỆU THAM KHẢO	71

DANH MỤC HÌNH VẼ

Hình 1. Bảng user.....	19
Hình 2. Bảng patient.....	20
Hình 3. Bảng doctor	21
Hình 4. Bảng Timeslot	22
Hình 5. Bảng hospital.....	23
Hình 6. Bảng department	24
Hình 7. Bảng MedicalVisit và Prescription	25
Hình 8. Bảng Appointment	27
Hình 9. Sơ đồ use case tổng quát	29
Hình 10. Sơ đồ tuần tự chức năng đăng ký	40
Hình 11. Sơ đồ tuần tự chức năng đăng nhập	41
Hình 12. Sơ đồ tuần tự chức năng quên mật khẩu	42
Hình 13. Sơ đồ tuần tự chức năng quên mật khẩu	43
Hình 14. Sơ đồ tuần tự chức năng đăng xuất	44
Hình 15. Sơ đồ tuần tự chức năng xem hồ sơ	44
Hình 16. Sơ đồ tuần tự chức năng xem lịch khám bệnh	45
Hình 17. Sơ đồ tuần tự chức năng cập nhật thông tin	46
Hình 18. Sơ đồ tuần tự chức năng đặt lịch khám bệnh	47
Hình 19. Sơ đồ tuần tự chức năng cập nhật trạng thái lịch hẹn	48
Hình 20. Sơ đồ tuần tự chức năng gửi thông báo.....	49
Hình 21. Sơ đồ lớp Singleton Pattern.....	50
Hình 22. Sơ đồ lớp Strategy Pattern.....	51

Hình 23. Sơ đồ lớp Template Method Pattern	52
Hình 24. Sơ đồ lớp Simple Factory Pattern	53
Hình 25. Sơ đồ lớp Command Pattern	55
Hình 26. Sơ đồ lớp Adapter Pattern	56
Hình 27. Sơ đồ lớp Observer Pattern	57
Hình 28. Sơ đồ lớp Decorator Pattern	58
Hình 29. Sơ đồ lớp State Pattern	59
Hình 30. Sơ đồ lớp Facade Pattern.....	60
Hình 31. Sơ đồ lớp Look/Guard Pattern	61
Hình 32. Sơ đồ lớp Global Exception Handler Pattern	62
Hình 33. Sơ đồ tổng quát tương tác giao diện bệnh nhân	64
Hình 34. Sơ đồ tổng quát tương tác giao diện bác sĩ.....	67

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Giới thiệu đề tài

Trong kỷ nguyên của cuộc Cách mạng Công nghiệp 4.0, sự giao thoa giữa y học hiện đại và các thành tựu công nghệ thông tin đã tạo ra một bước ngoặt mang tính lịch sử cho ngành y tế toàn cầu. Đề tài "**Xây dựng hệ thống ứng dụng dịch vụ y tế số Smart Healthcare**" không chỉ đơn thuần là việc phát triển một ứng dụng di động, mà là việc kiến tạo một hệ sinh thái chăm sóc sức khỏe thông minh, bền vững và lấy con người làm trung tâm. Hệ thống này được thiết kế để trở thành một "trợ lý y tế số" toàn năng, nơi các rào cản về khoảng cách địa lý, sự chậm trễ trong quy trình hành chính và sự rời rạc của dữ liệu y khoa được loại bỏ hoàn toàn. Thông qua việc số hóa toàn diện từ khâu tiếp nhận, chẩn đoán đến điều trị và theo dõi sau khám, đề tài hướng đến việc thiết lập một mạng lưới kết nối đa chiều, tương tác thời gian thực giữa bệnh nhân, đội ngũ y bác sĩ và các cơ sở y tế. Sự tích hợp của các nền tảng kỹ thuật số hiện đại không chỉ giúp tối ưu hóa nguồn lực xã hội mà còn đảm bảo rằng mọi cá nhân, bất kể vị trí địa lý hay điều kiện kinh tế, đều có cơ hội tiếp cận với những dịch vụ y tế chất lượng cao nhất một cách công bằng và minh bạch. Đây chính là lời giải cho bài toán hiện đại hóa nền y tế quốc gia, chuyển đổi từ mô hình vận hành thủ công sang mô hình quản trị số thông minh, đáp ứng kỳ vọng về một cuộc sống khỏe mạnh và tiện nghi trong tương lai.

1.2 Lý do chọn đề tài

Việc lựa chọn đề tài này xuất phát từ những bức bách thực tế của hệ thống y tế hiện nay, khi tình trạng quá tải tại các bệnh viện tuyến đầu đang trở thành một bài toán nan giải gây áp lực lớn lên cả đội ngũ y tế lẫn người bệnh. Quy trình khám chữa bệnh truyền thống với các thủ tục giấy tờ rườm rà, thời gian chờ đợi kéo dài và sự thiếu hụt trong việc kết nối dữ liệu giữa các bệnh viện đã dẫn đến sự lãng phí nguồn lực và giảm hiệu quả điều trị. Thêm vào đó, nhận thức của người dân về việc quản lý sức khỏe chủ động ngày càng cao, nhưng họ lại thiếu các công cụ khoa học để tự theo dõi và cảnh báo sớm các dấu hiệu bất thường. Sự bùng nổ của các thiết bị đeo thông

minh (IoT) và những đột phá trong lĩnh vực Trí tuệ nhân tạo (AI) đã tạo ra một tiền đề kỹ thuật vững chắc để giải quyết các vấn đề này. Tuy nhiên, hiện nay trên thị trường vẫn thiếu vắng một giải pháp tổng thể có khả năng tích hợp sâu các thiết bị này vào quy trình chuyên môn của bác sĩ. Do đó, việc xây dựng một hệ thống Smart Healthcare không chỉ mang ý nghĩa về mặt công nghệ mà còn là một nhiệm vụ cấp thiết về mặt xã hội, giúp giảm bớt gánh nặng kinh tế cho các hộ gia đình thông qua việc phòng bệnh từ xa, đồng thời hỗ trợ các cơ quan quản lý nhà nước trong việc hoạch định chính sách y tế dựa trên dữ liệu thực tế và chính xác.

1.3 Mục tiêu thực hiện đề tài

Mục tiêu cốt lõi của đề tài là nghiên cứu và phát triển một nền tảng dịch vụ y tế số toàn diện, có khả năng vận hành ổn định trên đa thiết bị, từ điện thoại thông minh đến máy tính bảng và các giao diện quản lý chuyên dụng. Dự án hướng tới việc thiết lập một hệ thống đặt lịch khám thông minh có khả năng tự động phân luồng bệnh nhân dựa trên triệu chứng sơ khởi, giúp giảm thiểu tối đa thời gian chờ đợi tại cơ sở y tế. Một mục tiêu quan trọng khác là triển khai mô hình tư vấn y tế từ xa (Telemedicine) với chất lượng hình ảnh và âm thanh đạt chuẩn y khoa, cho phép bác sĩ thăm khám và kê đơn trực tuyến cho các trường hợp bệnh lý nhẹ hoặc tái khám định kỳ. Về mặt dữ liệu, đề tài đặt mục tiêu xây dựng kho lưu trữ hồ sơ sức khỏe điện tử (EHR) cá nhân hóa, nơi mọi lịch sử bệnh án, kết quả xét nghiệm và đơn thuốc được số hóa và bảo mật tuyệt đối, giúp bệnh nhân dễ dàng tra cứu và bác sĩ có cái nhìn tổng thể về lộ trình bệnh lý của khách hàng. Xa hơn nữa, hệ thống kỳ vọng sẽ ứng dụng thành công các thuật toán Học máy (Machine Learning) để phân tích các chỉ số sinh tồn được gửi về từ thiết bị IoT, từ đó đưa ra các khuyến nghị dinh dưỡng, tập luyện hoặc cảnh báo sớm các nguy cơ đột quỵ, tim mạch, biến đổi chỉ số đường huyết, góp phần chuyển dịch mô hình chăm sóc sức khỏe từ ứng phó thụ động sang dự báo và phòng ngừa chủ động.

1.4 Phạm vi đề tài

Phạm vi nghiên cứu của đề tài được giới hạn tập trung vào hai trụ cột chính: giải pháp công nghệ và quy trình vận hành y khoa. Về mặt công nghệ, dự án tập trung phát triển ứng dụng di động trên hai hệ điều hành phổ biến nhất (iOS và Android) cùng với hệ thống quản trị nội dung (CMS) chạy trên nền tảng điện toán đám mây để đảm bảo khả năng mở rộng và truy cập linh hoạt. Đề tài sẽ đi sâu vào việc nghiên cứu các tiêu chuẩn bảo mật dữ liệu y tế quốc tế như HIPAA để đảm bảo quyền riêng tư tuyệt đối cho người dùng, đồng thời tích hợp các cổng thanh toán trực tuyến và hệ thống bảo hiểm xã hội để khép kín quy trình dịch vụ. Về mặt chuyên môn y tế, trong giai đoạn đầu, hệ thống sẽ tập trung vào các chuyên khoa phổ biến có nhu cầu tư vấn từ xa cao như nội tổng quát, nhi khoa, tâm lý và da liễu. Đối tượng thử nghiệm sẽ bao gồm một nhóm người dùng đại diện tại các đô thị lớn cùng với sự phối hợp của 3-5 phòng khám đa khoa có hạ tầng công nghệ thông tin cơ bản. Sau khi hoàn thiện mô hình và đánh giá hiệu quả thực tế, đề tài sẽ đề xuất lộ trình mở rộng quy mô, tích hợp sâu hơn với hệ thống quản lý bệnh viện (HIS) và mở rộng phạm vi ứng dụng ra các bệnh viện chuyên khoa và hệ thống y tế công cộng trên phạm vi toàn quốc.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Kiến trúc hướng dịch vụ

Kiến trúc hướng dịch vụ (SOA) là một phương pháp thiết kế phần mềm trong đó các ứng dụng được xây dựng từ các thành phần riêng biệt, gọi là các **dịch vụ (services)**. Mỗi dịch vụ là một đơn vị logic độc lập, thực hiện một chức năng kinh doanh cụ thể (ví dụ: dịch vụ đặt lịch, dịch vụ thanh toán, dịch vụ quản lý hồ sơ bệnh án) và giao tiếp với nhau qua các giao thức chuẩn.

Các đặc điểm chính của SOA bao gồm:

- **Tính lỏng lẻo (Loose Coupling):** Các dịch vụ ít phụ thuộc vào nhau về mặt kỹ thuật, cho phép thay đổi hoặc nâng cấp một dịch vụ mà không làm ảnh hưởng đến toàn bộ hệ thống.
- **Tính tái sử dụng (Reusability):** Một dịch vụ (như xác thực người dùng) có thể được sử dụng bởi nhiều module khác nhau trong hệ thống Smart Healthcare.
- **Tính trừu tượng (Abstraction):** Người dùng dịch vụ chỉ cần biết cách gọi dịch vụ (giao diện - interface) mà không cần quan tâm đến cách nó được lập trình bên trong.

Ứng dụng SOA vào hệ thống y tế số mang lại những lợi ích vượt trội:

- **Tích hợp hệ thống không đồng nhất:** Y tế là lĩnh vực có nhiều hệ thống cũ (legacy systems). SOA đóng vai trò như một lớp trung gian giúp ứng dụng di động mới có thể giao tiếp mượt mà với các cơ sở dữ liệu bệnh viện cũ thông qua các Web Services (RESTful hoặc SOAP).
- **Khả năng mở rộng linh hoạt:** Khi lượng người dùng tăng đột biến (ví dụ trong mùa dịch), hệ thống có thể chỉ cần nâng cấp các dịch vụ chịu tải cao (như dịch vụ tư vấn trực tuyến) thay vì phải nâng cấp toàn bộ hạ tầng.

- **Hỗ trợ đa nền tảng:** Do giao tiếp qua các giao thức web chuẩn, dữ liệu y tế có thể hiển thị đồng bộ trên ứng dụng di động, trang web và cả các thiết bị IoT theo dõi sức khỏe.

2.2 Mẫu thiết kế

Mẫu thiết kế (Design Pattern) là các giải pháp tổng quát đã được tối ưu hóa cho các vấn đề phổ biến trong thiết kế phần mềm. Đối với một hệ thống phức tạp như y tế số — nơi cần sự chính xác tuyệt đối và khả năng kết nối đa dạng — việc áp dụng Design Patterns giúp chuẩn hóa cấu trúc dữ liệu và quy trình xử lý giữa các tầng ứng dụng.

Lợi ích của việc áp dụng Design Patterns trong đề tài

- **Tính linh hoạt:** Dễ dàng thêm mới các loại cảm biến sức khỏe hoặc các phương thức thanh toán mà không làm phá vỡ cấu trúc hiện tại.
- **Khả năng bảo trì:** Mã nguồn được tổ chức theo các khuôn mẫu chuẩn giúp đội ngũ phát triển dễ dàng đọc hiểu và sửa lỗi.
- **Tối ưu hóa hiệu suất:** Giảm thiểu việc khởi tạo đối tượng dư thừa và tối ưu hóa luồng giao tiếp giữa các thành phần trong hệ thống Smart Healthcare.

2.3 Công nghệ sử dụng

Giao diện front-end: dùng ngôn ngữ lập trình Java xây dựng giao diện dạng ứng dụng di động trên chạy trên hệ điều hành Android.

Hệ thống back-end: xây dựng hệ thống back-end Restful API với framework Java Spring Boot.

Database: cơ sở dữ liệu được thiết riêng cho từng service với ngôn ngữ MySQL.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

3.1 Phân tích kiến trúc hệ thống

Hệ thống được thiết kế theo kiến trúc Microservices hiện đại, chia nhỏ các chức năng nghiệp vụ thành các dịch vụ độc lập, tự chủ và giao tiếp với nhau thông qua các giao thức chuẩn. Kiến trúc này giúp hệ thống Smart Healthcare đảm bảo tính sẵn sàng cao và khả năng mở rộng linh hoạt khi số lượng người dùng tăng đột biến.

Hệ thống được chia thành 4 lớp chính:

- Lớp Trình diễn: Bao gồm ứng dụng di động cho bệnh nhân, cho bác sĩ và Dashboard quản trị.
- Lớp Gateway: Điểm tiếp nhận duy nhất cho mọi yêu cầu từ phía Client. Chịu trách nhiệm định tuyến, xác thực, phân quyền và kiểm soát lưu lượng.
- Lớp Dịch vụ: Tập hợp các dịch vụ nghiệp vụ cốt lõi, hoạt động độc lập.
- Lớp Dữ liệu: Mỗi Microservice sở hữu cơ sở dữ liệu riêng (Database-per-service) để đảm bảo tính bao đóng.

Hệ thống được chia thành các dịch vụ chính sau:

- User-service: Nghiệp vụ liên quan đến người dùng(xác thực, phân quyền, ...).
- Patient-service: Thao tác đọc, ghi, xoá, sửa dữ liệu của người dùng có vai trò là bệnh nhân.
- Appointment-service: Các chức năng nghiệp vụ liên quan đến đặt lịch hẹn.
- Hospital-service: Các thao tác đọc, ghi, xoá, sửa dữ liệu về bệnh viện (hospital) và khoa (department).
- Doctor-service: Thao tác đọc, ghi, xoá, sửa dữ liệu của người dùng có vai trò là bác sĩ.
- EHR-service: Thao tác đọc, ghi, sửa, xoá dữ liệu liên quan đến đơn thuốc, lịch sử khám bệnh.

- Notification-service: Dành riêng cho việc gửi các thông báo bằng email đến người dùng là bệnh nhân hoặc bác sĩ.
- Api-gateway-service: là trung tâm điều phối nhận các request và điều phối các request đến service thích hợp.
- Các thư viện dùng chung trong hệ thống: common-exception:

Để đảm bảo dữ liệu y tế luôn chính xác trong môi trường phân tán, hệ thống áp dụng:

- Database-per-service: Mỗi dịch vụ sử dụng loại DB phù hợp (ví dụ: Patient Service dùng MongoDB cho dữ liệu phi cấu trúc, Booking Service dùng PostgreSQL cho tính toán vụn dữ liệu).
- Saga Pattern: Quản lý các giao dịch phân tán giữa các dịch vụ (ví dụ: quy trình đặt lịch và thanh toán) để đảm bảo dữ liệu được hoàn tác (Rollback) nếu có lỗi xảy ra ở bất kỳ mắt xích nào.

3.2 Phân tích vai trò người dùng

3.2.1 Bệnh nhân

Bệnh nhân là người sử dụng chính của ứng dụng. Bệnh nhân phải truy cập vào hệ thống bằng một tài khoản và thực hiện khai báo một số thông tin cần thiết. Sau đó, bệnh nhân có thể đặt lịch khám của bác sĩ và xem lại lịch mình đã đặt.

3.2.2 Bác sĩ

Bác sĩ là người phụ trách khám bệnh cho bệnh nhân, mỗi bác sĩ phải cũng cấp đầy đủ các thông tin cá nhân cơ bản đủ để bệnh nhân có thể nhận biết được bác sĩ nào và thông tin này sẽ được hệ thống hiện lên khi bệnh nhân thực hiện tìm bác sĩ. Bác sĩ cũng có thể xem lịch làm việc của mình.

3.2.3 Quản trị viên hệ thống

Quản trị viên là người có tất cả các quyền thêm, sửa, xóa thông tin các toàn bộ hệ thống, quản trị viên phải đảm bảo hệ thống hoạt động trơn tru, hiệu quả.

CHƯƠNG 4. YÊU CẦU HỆ THỐNG

4.1 Yêu cầu chức năng

4.1.1 Xác thực người dùng

4.1.1.1 Đăng nhập

Mục tiêu: cho phép người dùng đã đăng ký truy cập vào các tính năng cá nhân hoá và quản trị hệ thống bằng cách xác thực danh tính.

Người thực hiện: bác sĩ, bệnh nhân, quản trị viên.

Các phương thức đăng nhập:

- Dùng email và mật khẩu: người dùng nhập email và mật khẩu đã đăng ký để đăng nhập.
- Dùng số định danh cá nhân và mật khẩu: người dùng nhập thông tin số định danh cá nhân để đăng nhập.
- Dùng số điện thoại và mật khẩu: người dùng nhập số điện thoại và mật khẩu đã đăng ký để thực hiện đăng nhập.

Quy trình:

- Tại trang đăng nhập người dùng nhập username(email, số điện thoại hoặc cccd) và mật khẩu sau đó nhấn nút đăng nhập.
- Hệ thống sẽ thực hiện gửi thông tin lên server để xác thực người dùng và trả lại kết quả.
- Nếu đăng nhập không thành công(sai username hoặc mật khẩu) hệ thống sẽ thông báo lỗi đăng nhập kèm theo nguyên nhân và yêu cầu người dùng thực hiện lại quy trình.
- Nếu đăng nhập thành công (đúng username và mật khẩu) hệ thống sẽ kiểm tra vai trò và điều hướng người dùng đến trang làm việc của mình.

4.1.1.2 Đăng ký

Mục tiêu: Cho phép người dùng đăng ký tài khoản mới trên hệ thống để thực hiện các chức năng yêu cầu định danh.

Người thực hiện: quản trị viên, bệnh nhân.

Các thông tin cần nhập và xác thực thông tin:

- Số căn cước công dân: yêu cầu phải có đủ 12 số và không được phép có chữ và không được rỗng.
- Email: phải bao gồm ký tự “@gmail.com” và không được rỗng.
- Số điện thoại: số điện thoại phải có 10 số và không được rỗng.
- Tên người dùng: không được rỗng.
- Địa chỉ: không được rỗng.
- Ngày sinh: ngày sinh phải có giá trị bé hơn hiện tại và không được rỗng.
- Giới tính: giới tính bao gồm nam và nữ, không được rỗng.

Bên cạnh đó người dùng cũng phải nhập những thông tin về sức khỏe cá nhân cơ bản bao gồm:

- Số bảo hiểm y tế: được phép trống.
- Số điện thoại khẩn cấp: số điện thoại khẩn cấp khác số điện thoại của người đăng ký, và phải bao gồm 10 số, không được để trống.
- Công việc: được phép trống.
- Cân nặng và chiều cao: phải là số và không được trống.
- Nhóm máu: được phép trống.

Quy trình:

- Khi người dùng chọn đăng ký tài khoản hệ thống sẽ chuyển sang trang cho phép người dùng nhập thông tin cá nhân.
- Sau khi nhập xong người dùng nhấp nút tạo tài khoản, hệ thống điều hướng đến trang nhập thông tin bổ sung cho bệnh nhân.
- Sau khi người dùng nhập hoàn tất các thông tin, lúc này hệ thống sẽ thực hiện xác thực các thông tin do người dùng nhập. Sau khi kiểm tra, thông tin được gửi tới server lúc này server kiểm tra tính hợp lệ của trường thông tin căn cước công dân, email và số điện thoại. Nếu 1 trong 3 trường thông tin này bị trùng lặp sẽ báo lỗi và không được tạo tài khoản. Hệ thống sẽ gửi thông báo lên phía người dùng để kiểm tra lại thông tin.

- Nếu quá trình đăng ký thất bại hệ thống sẽ dừng lại ở trang nhập thông tin và hiển thị thông báo lỗi.
- Nếu quá trình đăng ký thành công hệ thống sẽ chuyển người dùng vào trang chủ của ứng dụng và gửi đi một email thông báo đã đăng ký tài khoản thành công cho email đã thực hiện đăng ký.

4.1.1.3 Quên mật khẩu

Mục tiêu: cung cấp phương thức thay đổi mật khẩu an toàn cho người dùng có thể thiết lập lại mật khẩu trong trường hợp mất mật khẩu.

Người thực hiện: bác sĩ, bệnh nhân, quản trị viên.

Thông tin cần nhập: và mã OTP

Quy trình:

- Khi người dùng chọn quên mật khẩu, hệ thống sẽ chuyển sang trang yêu cầu của người dùng nhập email.
- Sau khi người dùng nhập email, hệ thống thực hiện kiểm tra email có tồn tại hay không và thực hiện gửi mã OTP đến email này. Mã OTP này chỉ có hiệu lực trong thời gian 5 phút.
- Người dùng nhập mã OTP để xác thực danh tính. Nếu nhập sai hệ thống báo lỗi và chỉ cho phép nhập sai 5 lần. Nếu nhập chính xác hệ thống chuyển người dùng đến trang nhập mật khẩu mới.
- Sau khi người dùng nhập mật khẩu mới hệ thống sẽ thực hiện cập nhật mật khẩu mới cho người dùng và sẽ lưu ở trạng thái bấm để đảm bảo an toàn và thông báo đổi mật khẩu thành công. Sau đó hệ thống chuyển về trang đăng nhập.

4.1.1.4 Thay đổi mật khẩu

Mục tiêu: Cho phép người dùng cập nhật mật khẩu mới một cách an toàn để bảo vệ tài khoản với yêu cầu phải xác thực chính xác mật khẩu hiện tại.

Người thực hiện: bác sĩ, bệnh nhân, quản trị viên.

Thông tin cần nhập: mật khẩu hiện tại, mật khẩu mới và xác nhận mật khẩu mới.

Quy trình: để thực hiện được tính năng này yêu cầu người dùng đang đăng nhập.

- Tại trang thông tin người dùng, người dùng chọn biểu tượng thay đổi mật khẩu, hệ thống chuyển người dùng đến trang thay đổi mật khẩu.
- Người dùng phải nhập đầy đủ các thông tin bao gồm(mật khẩu hiện tại, mật khẩu mới và xác nhận mật khẩu mới). Sau đó nhấn thay đổi.
- Lúc này hệ thống sẽ thực hiện kiểm tra thông tin của mật khẩu hiện tại nếu sai hệ thống báo lỗi và yêu cầu người dùng nhập lại; nếu đúng hệ thống tiếp tục kiểm tra mật khẩu mới và xác nhận mật khẩu mới nếu 2 trường thông tin này trùng nhau thì thực hiện cập nhật mật khẩu mới vào cơ sở dữ liệu dưới dạng băm. Sau đó, thông báo thay đổi mật khẩu thành công và chuyển người dùng về trang chính.

4.1.2 Đặt lịch khám bệnh

Mục tiêu: Cung cấp quy trình trực tuyến cho phép bệnh nhân chủ động lựa chọn bác sĩ, chuyên khoa và khung giờ khám phù hợp; tự động hóa việc quản lý lịch trình của bác sĩ, tránh tình trạng trùng lịch và giảm thiểu thời gian chờ đợi tại bệnh viện; đảm bảo thông tin lịch hẹn được lưu trữ đồng bộ giữa các hệ thống.

Người thực hiện: bệnh nhân.

Các thông tin:

- Thông tin bệnh nhân: tên, số điện thoại người đặt lịch
- Thông tin bác sĩ và cơ sở khám bệnh: Tên bác sĩ, tên cơ sở khám bệnh, tên khoa.
- Thông tin thời gian: Ngày, giờ chính thức đi khám.
- Thông tin bổ sung: lý do, ghi chú.

Quy trình:

- Tìm kiếm và lựa chọn: Bệnh nhân tìm kiếm bác sĩ theo Chuyên khoa và bệnh viện.

- Chọn khung giờ: Hệ thống hiển thị danh sách các khung giờ trống của bác sĩ.
- Khởi tạo lịch hẹn: Bệnh nhân nhập lý do khám và xác nhận đặt chỗ. Hệ thống tạo bản ghi Appointment.
- Cập nhật trạng thái khung giờ: Ngay sau khi lịch hẹn được tạo, hệ thống cập nhật isAvailable = false cho TimeSlot tương ứng để ngăn người khác đặt trùng.
- Xác nhận: Hệ thống gửi thông báo xác nhận lịch hẹn cho cả bệnh nhân và bác sĩ qua email hoặc ứng dụng.
- Hoàn tất: Lịch hẹn được lưu vào cơ sở dữ liệu và chờ đến ngày thăm khám để chuyển sang trạng thái COMPLETED khi bác sĩ xác nhận hoàn thành hoặc CANCELLED khi bác sĩ hoặc bệnh nhân xác nhận huỷ lịch.

4.1.3 Cập nhật trạng thái lịch khám bệnh

Mục tiêu: quản lý vòng đời của một lịch hẹn từ khi được tạo cho đến khi kết thúc hoặc bị hủy, đảm bảo tính chính xác về dữ liệu giữa lịch làm việc thực tế của bác sĩ và trạng thái trên hệ thống, kịp thời giải phóng các khung giờ khám nếu lịch hẹn bị hủy để các bệnh nhân khác có thể đặt chỗ.

Người thực hiện: bệnh nhân và bác sĩ.

Các thông tin:

- Thông tin trạng thái: Trạng thái mới cần chuyển sang.
- Thời gian ghi nhận: thời gian cập nhật và thời gian chờ.
- Lý do thay đổi: Ghi chú lý do hủy lịch từ phía bác sĩ hoặc bệnh nhân(nếu có).

Quy trình:

- Tiếp nhận yêu cầu: Hệ thống nhận yêu cầu thay đổi trạng thái từ phía Bệnh nhân, Bác sĩ (Xác nhận/Hoàn thành) hoặc từ hệ thống tự động.
- Kiểm tra điều kiện: Hệ thống kiểm tra xem trạng thái hiện tại có hợp lệ để chuyển sang trạng thái mới hay không.

- Cập nhật trường status trong bảng Appointment.
- Tự động cập nhật updatedAt để lưu vết thời gian thay đổi.
- Nếu trạng thái là CANCELLED: Hệ thống tự động tìm đến timeSlotId liên quan và cập nhật lại isAvailable = true để mở lại khung giờ cho người khác.
- Nếu trạng thái là COMPLETED: Hệ thống đánh dấu lịch hẹn đã kết thúc, sẵn sàng để bác sĩ tạo hồ sơ MedicalVisit.
- Thông báo: Gửi thông báo tự động cho các bên liên quan về việc trạng thái lịch hẹn đã thay đổi thành công.

4.1.4 Xem hồ sơ sức khỏe

Mục tiêu: Cho phép người dùng xem lại các thông tin họ đã cung cấp cho hệ thống xem đã chính xác chưa, đặc biệt là các thông tin nhạy cảm như căn cước công dân, số điện thoại. Nơi để người dùng có thể thực hiện các chức năng khác như chỉnh sửa, đăng xuất, thay đổi mật khẩu.

Người thực hiện: bác sĩ, bệnh nhân, quản trị viên.

Các thông tin hiển thị: Họ và tên, số căn cước công dân, số điện thoại, email, ngày tháng năm sinh, vai trò, địa chỉ, số bảo hiểm y tế, chiều cao, cân nặng, nhóm máu.

Quy trình:

- Khi người dùng chọn chức năng xem thông tin cá nhân trên màn hình chính của ứng dụng, hệ thống sẽ chuyển người dùng đến trang thông tin cá nhân.
- Tại đây hệ thống sẽ hiển thị các thông tin cá nhân được nêu trên và các chức năng: thay đổi thông tin cá nhân, đăng xuất, thay đổi mật khẩu.

4.1.5 Xem danh sách lịch khám bệnh

Mục tiêu: cho phép bác sĩ theo dõi lịch trình làm việc hàng ngày, hàng tuần để chuẩn bị công tác khám chữa bệnh; giúp bệnh nhân quản lý các lịch hẹn đã đặt, kiểm tra trạng thái và thời gian khám để sắp xếp công việc cá nhân; tăng cường tính minh bạch và quản lý dữ liệu lịch hẹn tập trung.

Người thực hiện: bác sĩ và bệnh nhân.

Các thông tin:

- Mã lịch hẹn: appointmentId.
- Thông tin đối tác: tên bác sĩ (cho bệnh nhân) hoặc tên bệnh nhân(cho bác sĩ).
- Địa điểm: tên khoa, tên bệnh viện thực hiện khám.
- Thời gian: thời gian thực hiện khám.
- Trạng thái: trạng thái của lịch hẹn.
- Nội dung: lý do, ghi chú.

Quy trình:

- Gửi yêu cầu: Người dùng truy cập vào mục "Lịch hẹn của tôi" (đối với bệnh nhân) hoặc "Lịch khám sắp tới" (đối với bác sĩ).
- Truy vấn dữ liệu: Hệ thống gửi yêu cầu đến Appointment Service kèm theo mã định danh người dùng (patientId hoặc doctorId).
- Lọc dữ liệu (Tùy chọn): Người dùng có thể chọn lọc theo trạng thái (ví dụ: chỉ xem lịch "CONFIRMED") hoặc lọc theo ngày tháng cụ thể.
- Xử lý và hiển thị: Hệ thống sắp xếp danh sách theo thời gian gần nhất (appointmentDateTime) và hiển thị lên giao diện người dùng.
- Tương tác: Người dùng có thể nhấn vào một bản ghi cụ thể để xem chi tiết hoặc thực hiện các thao tác phụ như "Hủy lịch" (nếu là bệnh nhân) hoặc "Cập nhật trạng thái" (nếu là bác sĩ).

4.1.6 Thông báo

4.1.6.1 Gửi thông báo đến email khi người dùng tạo tài khoản thành công.

Mục tiêu: Nhằm xác nhận việc khởi tạo tài khoản thành công, cung cấp những thông tin cơ bản cho dùng và hướng dẫn bước tiếp theo cần phải thực hiện.

Người thực hiện: hệ thống tự động gửi.

Các thông tin cần gửi trong email: Tên người dùng, số điện thoại, số căn cước công dân.

Quy trình: Sau khi người dùng tạo thành công tài khoản hệ thống sẽ thực hiện gửi một email đến người dùng thông báo việc tạo tài khoản thành công.

4.1.6.2 Gửi thông báo khi người dùng đặt lịch hẹn thành công.

Mục đích: Xác nhận lần nữa với người dùng về lịch hẹn đã được đặt thành công, cung cấp phương thức tra cứu nhanh khi người dùng đến khám tại cơ sở y tế, cung cấp các lưu ý quan trọng trước khi người dùng đến khám.

Các thông tin cần gửi trong email: người nhận lịch hẹn, mã lịch hẹn, trạng thái, thời gian, địa điểm và các lưu ý khi đến khám.

Quy trình: Khi người dùng đặt lịch hẹn hoàn tất hệ thống sẽ gửi một email chứa trạng thái của lịch hẹn và các thông tin cần thiết để người dùng có thể nắm thêm về lịch hẹn. Email chỉ được gửi khi người dùng đặt lịch hẹn thành công.

4.1.6.3 Hiện thông báo trong ứng dụng về cuộc hẹn vừa đặt.

Mục đích: Thông báo cho người dùng về trạng thái của cuộc hẹn vừa đặt ngay trong ứng dụng, nhắc nhở người dùng kiểm tra lại trạng thái lịch hẹn của mình.

Các thông tin cần hiện: người nhận lịch hẹn, mã lịch hẹn, trạng thái, thời gian, địa điểm.

Quy trình:

- Ghi nhận thành công: Người dùng hoàn tất đặt lịch, ứng dụng nhận được phản hồi (Response 200 OK) từ Server.
- Cập nhật trạng thái cục bộ: Ứng dụng lưu trạng thái vào bộ nhớ tạm (SharedPreferences).
- Hiển thị chấm đỏ: Giao diện chính tại mục lịch sử cuộc hẹn sẽ hiển thị một chấm màu đỏ thông báo có lịch hẹn mới.
- Đọc và xóa: Khi người dùng bấm vào tab "Lịch sử", ứng dụng tải danh sách lịch hẹn mới nhất. Ngay lúc đó, chấm đỏ biến mất khỏi màn hình.

4.1.6.4 Thông báo cho bác sĩ về lịch hẹn mới của mình.

Mục đích: Giúp bác sĩ nắm bắt thông tin kịp thời khi có bệnh nhân đặt lịch khám thành công. Từ đó, bác sĩ có thể chủ động quản lý lịch trình làm việc của mình, xem trước trước lý do khám để có sự chuẩn bị tốt nhất.

Các thông tin: tên người đặt, tên bệnh viện, tên khoa, lý do.

Quy trình: Ngay sau khi bệnh nhân đặt lịch hẹn thành công và nhận trạng thái CONFIRM lúc này hệ thống sẽ gửi một email xác nhận tới bác sĩ nhận lịch khám.

4.1.7 Thay đổi thông tin cá nhân

Mục tiêu: Cho phép người dùng cập nhật chỉnh sửa thông tin cơ bản của bản thân để phù hợp với thực tế, phục vụ cho việc liên lạc và nhận diện trên hệ thống. Yêu cầu chức năng phải được thực hiện khi người dùng đã đăng nhập vào hệ thống. Quá trình thay đổi thông tin đảm bảo thông tin từ trạng thái phù hợp này sang trạng thái phù hợp khác và không được phép sai.

Các thông tin được phép thay đổi: Số căn cước công dân, email, số điện thoại, ngày sinh, tên, địa chỉ, giới tính, số bảo hiểm, số điện thoại khẩn cấp, cân nặng, chiều cao, nhóm máu.

Quy trình:

- Tại trang thông tin người dùng, người dùng chọn biểu tượng thay đổi thông tin, lúc này hệ thống chuyển người dùng đến trang thay đổi thông tin.
- Lúc này, hệ thống sẽ hiện những thông tin cũ của người dùng lên và người dùng được phép thực hiện chỉnh sửa cho phù hợp.
- Khi người dùng chọn cập nhật, lúc này hệ thống sẽ kiểm tra lại tính hợp lý của thông tin giống như lúc đăng ký. Nếu tất cả các thông tin đều hợp lý thì thực hiện cập nhật vào cơ sở dữ liệu. Nếu có bất kì thông tin nào đó trùng lặp hoặc không chính xác hệ thống sẽ thông báo lỗi và dừng cập nhật.
- Sau khi quá trình cập nhật thành công hệ thống sẽ hiện thông báo và chuyển người dùng về trang thông tin người dùng.

4.2 Yêu cầu phi chức năng

4.2.1 Bảo mật và Riêng tư

- Mã hóa: Toàn bộ dữ liệu bệnh nhân phải mã hóa chuẩn khi lưu
- Xác thực: Bắt buộc đăng nhập đối với người dùng.
- Truy vết (Audit Trail): Ghi nhật ký vĩnh viễn mọi thao tác xem/sửa hồ sơ bệnh án, không được phép xóa log.

4.2.2 Độ tin cậy & Sẵn sàng

- Uptime: Hệ thống hoạt động ổn định.
- Có thể đáp ứng khi số lượng người dùng tăng cao.

4.2.3 Hiệu năng

- Tốc độ phản hồi: Các truy vấn hồ sơ thông thường phải hiển thị trong dưới 2 giây.
- Chịu tải: Hệ thống xử lý mượt mà khi số lượng người dùng tăng cao.

CHƯƠNG 5. PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU

5.1 Phân tích cơ sở dữ liệu

Thay vì xây dựng toàn bộ hệ thống thành một khối monolithic khổng lồ, hệ thống Smart-Healthcare được thiết kế theo kiến trúc Microservice với ngôn ngữ thiết kế mỗi service có một cơ sở dữ liệu riêng.

Các lợi ích khi xây dựng hệ thống dựa trên kiến Microservice:

- Tính bảo mật: Mỗi service chỉ được truy cập vào cơ sở dữ liệu của service đó, nếu muốn lấy dữ liệu từ cơ sở dữ liệu của service khác phải thông qua API, điều này đảm bảo tính đóng gói cho toàn bộ hệ thống.
- Tối ưu hoá công nghệ: Vì mỗi service có một cơ sở dữ liệu riêng nên có thể linh hoạt lựa chọn hệ cơ sở dữ liệu phù hợp theo nguyên tắc nghiệp vụ của từng service.

Do hệ mỗi service được kết nối với một cơ sở dữ liệu khác nhau nên sẽ không tồn tại những liên kết khoá ngoại giữa các bảng trên toàn hệ thống. Do đó, tại cơ sở dữ liệu của các bảng có liên kết với nhau sẽ được thiết kế thêm một trường dữ liệu dùng để nhận diện và tham chiếu đến bảng thuộc cơ sở dữ liệu khác.

Tuy nhiên, để tối ưu khả năng truy vấn, thay vì chỉ lưu 1 trường dữ liệu tham chiếu, hệ thống có thể sẽ dùng nhiều hơn 1 trường dữ liệu, nhưng tính nhất quán dữ liệu vẫn sẽ được đảm bảo.

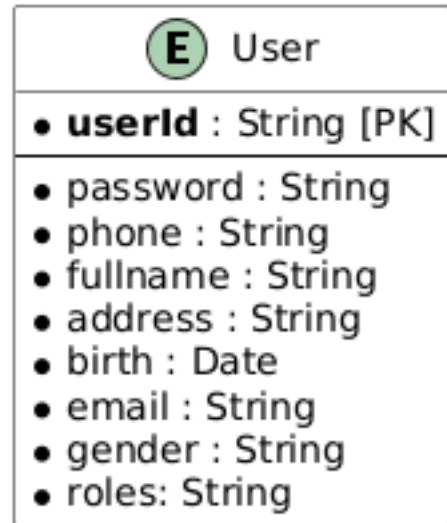
5.2 Thiết kế dữ liệu cho các service

5.2.1 User service

Chịu trách nhiệm lưu trữ thông tin toàn bộ người dùng trong hệ thống. Chức năng chính của service này là đảm bảo việc đăng nhập, đăng ký, phân quyền, và các tính năng hỗ trợ về thông tin người dùng trong toàn bộ hệ thống.

Thiết kế dữ liệu cho user-service bao gồm các trường:

- **userId**: số căn cước công dân, đây là trường khoá chính trong bảng này.
- **password**: mật khẩu của người dùng.
- **phone**: số điện thoại
- **fullname**: Tên đầy đủ
- **address**: địa chỉ hiện tại
- **birth**: ngày sinh
- **email**: email của người dùng
- **gender**: giới tính
- **roles**: Vai trò trong hệ thống



Hình 1. Bảng user

Các quy tắc nghiệp vụ:

- **userId, phone, email**: các trường dữ liệu này phải là duy nhất cho toàn hệ thống. Các người dùng không được phép trùng ở trường dữ liệu này.
- **gender**: trường dữ liệu này chỉ nhận 2 giá trị duy nhất là “MALE” và “FEMALE”.
- **roles**: trường dữ liệu này cho phép một người dùng có thể có nhiều vai trò. Trường dữ liệu này chỉ nhận các dữ liệu bao gồm: “PATIENT”, “DOCTOR”, “ADMIN”.

5.2.2 Patient service

Service này chịu trách nhiệm lưu trữ toàn bộ thông tin liên quan đến các người dùng có vai trò là bệnh nhân trong hệ thống. Chức năng chính của service này dùng để lưu trữ thông tin và đổ thông tin vào EHR. Bên cạnh đó, bệnh nhân có quyền thay đổi một số thông tin cơ của mình.

Thiết kế dữ liệu cho patient-service bao gồm các trường:

- **patientId**: Mã định danh duy nhất của bệnh nhân (khóa chính).
- **fullName**: Họ và tên đầy đủ.
- **userId**: ID liên kết với tài khoản người dùng trong hệ thống.
- **birth**: Ngày, tháng, năm sinh.
- **registeredAt**: Thời điểm bệnh nhân đăng ký hồ sơ lần đầu.
- **gender**: Giới tính của bệnh nhân.
- **insuranceId**: Mã số thẻ bảo hiểm y tế.
- **emergencyCallingNumber**: Số điện thoại liên lạc trong tình huống khẩn cấp.
- **job**: Nghề nghiệp của bệnh nhân.
- **bloodType**: Nhóm máu.
- **heights**: Chiều cao.
- **weights**: Cân nặng.
- **isActive**: Trạng thái hồ sơ.

E Patient
• patientId : String [PK] • fullName : String • userId : String • birth : Date • registeredAt : Date • gender : String • insuranceId : String • emergencyCallingNumber : String • job : String • bloodType : String • heights : Double • weights : Double • isActive : Boolean

Hình 2. Bảng patient

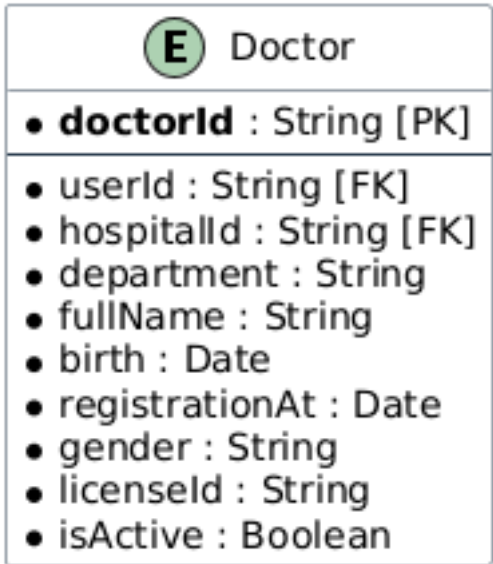
Các quy tắc nghiệp vụ:

- **userId**: đây là trường dữ liệu dùng để tham chiếu đến dữ liệu trong bảng **user** của **user-service**. Do đó, để tồn tại **patient** thì phải tồn tại **user**.
- **fullname, birth, gender**: Các trường dữ liệu này phải được đồng bộ với người dùng có **userId** được lưu ở trên.
- **isActive**: trường dữ liệu này quyết định người dùng có hoạt động hay không.
- **bloodType**: trường dữ liệu này chỉ nhận dữ liệu bao gồm: “A”, “AB”, “B”, “O”.

5.2.3 Doctor service

Đây là service chịu trách nhiệm lưu các thông tin của đối tượng người dùng là bác sĩ. Bên cạnh việc lưu thông tin của bác sĩ, service này chịu trách nhiệm lưu thông tin các lịch hẹn của bác sĩ trong bảng `TimeSlots`. Bác sĩ được quyền sửa các thông tin cá nhân cơ bản của mình bằng thông qua phương được cung cấp.

Thiết kế dữ liệu trong doctor-service bao gồm 2 bảng:

Bảng doctors	
- <code>doctorId</code>: Mã định danh duy nhất của bác sĩ (Khóa chính). - <code>userId</code>: ID liên kết với tài khoản người dùng của bác sĩ. - <code>hospitalId</code>: ID của bệnh viện hoặc cơ sở y tế nơi bác sĩ đang công tác. - <code>department</code>: Mã khoa đang công tác. - <code>fullName</code>: Họ và tên đầy đủ. - <code>birth</code>: Ngày, tháng, năm sinh. - <code>registrationAt</code>: Thời điểm đăng ký bác sĩ. - <code>gender</code>: Giới tính của bác sĩ. - <code>licenseId</code>: Số giấy phép hành nghề y tế của bác sĩ. - <code>isActive</code>: Trạng thái hồ sơ.	

Hình 3. Bảng doctor

Các quy tắc nghiệp vụ của bảng doctors:

- `userId`: đây là trường dữ liệu dùng để tham chiếu đến dữ liệu của bác sĩ trong bảng `users` của `user-service`. Để tồn tại dữ liệu trong bảng `doctors` thì phải tồn tại dữ liệu trong bảng `users`.

- `fullname`, `birth`, `gender`: đây là các trường dữ liệu nhận lấy từ bảng `user`. Các dữ liệu này phải đồng nhất với dữ liệu trong bảng `users` khi có thực hiện chỉnh sửa.
- `isActive`: trường dữ liệu này quy định bác sĩ còn làm việc tại bệnh viện hay tại khoa không.
- `hospitalId`: trường dữ liệu dùng để tham chiếu đến bảng `hospitals` trong `hospital-service`. Trường dữ liệu này không được để trống.
- `department`: trường dữ liệu dùng để tham chiếu đến bảng `departments` trong `hospital-service`. Trường dữ liệu này không được trống.

Bảng Timeslots

- `timeSlotId`: Mã định danh duy nhất cho khung giờ (khóa chính).
- `doctorId`: ID của bác sĩ.
- `dayOfWeek`: Thứ trong tuần.
- `startTime`: Giờ bắt đầu của ca khám.
- `endTime`: Giờ kết thúc của ca khám.
- `isAvailable`: Trạng thái.
- `specificDate`: Ngày cụ thể.
- `createdAt`: Thời điểm khung giờ này được tạo ra trên hệ thống.

E TimeSlot
• timeSlotId : String [PK] «UUID»
• doctorId : String [FK] • dayOfWeek : String • startTime : LocalTime • endTime : LocalTime • isAvailable : Boolean • specificDate : LocalDate • createdAt : LocalDateTime

Hình 4. Bảng Timeslot

Các quy tắc nghiệp vụ của bảng Timeslots:

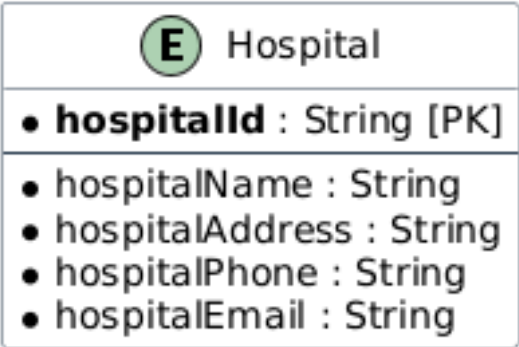
- `doctorId`: trường dữ liệu này tham chiếu đến đối tượng bác sĩ thuộc bảng bác sĩ. Để `TimeSlot` này tồn tại thì dữ liệu `doctorId` này phải tồn tại.
- `dayOfWeek`: dữ liệu này mô tả ngày trong tuần. Chỉ nhận dữ liệu trong danh sách sau: “MONDAY”, “TUESDAY”, “WEDNESDAY”, “THURSDAY”, “FRIDAY”, “SATURDAY”, “SUNDAY”.

- `isAvailable`: trường dữ liệu này quyết định một lịch khám đã được đặt hay chưa. Nếu `False` là đã được đặt, ngược lại là còn trống.

5.2.4 Hospital service

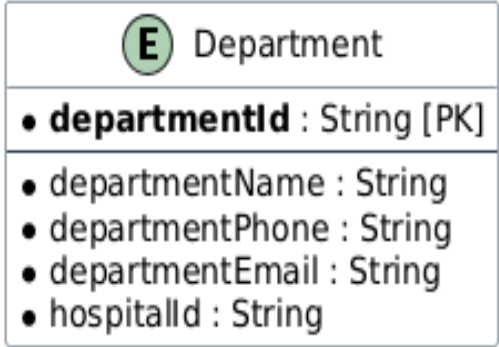
Service này quản lý toàn bộ thông tin của các bệnh viện nằm trong hệ thống Smart-Healthcare. Thông thường chỉ có admin được phép tương tác đến các thông tin. Dữ liệu được lưu thành hai bảng là `Hospitals`-Lưu thông tin các bệnh viện trong toàn hệ thống, `Departments`-Lưu thông tin các khoa của một bệnh viện. Các chức năng chỉnh sửa, thêm, xoá chỉ được admin của hệ thống thực hiện.

Thiết kế dữ liệu cho bảng `Hospital` như sau:

Bảng Hospitals	
- <code>hospitalId</code>: Mã định danh duy nhất của bệnh viện. - <code>hospitalName</code>: Tên của bệnh viện. - <code>hospitalAddress</code>: Địa chỉ của bệnh viện. - <code>hospitalPhone</code>: Tổng đài chăm sóc khách hàng của bệnh viện. - <code>hospitalEmail</code>: Địa chỉ thư điện tử của bệnh viện.	 <pre> classDiagram class Hospital { hospitalId String PK hospitalName String hospitalAddress String hospitalPhone String hospitalEmail String } </pre>

Hình 5. Bảng hospital

Thiết kế dữ liệu cho bảng Departments:

Bảng Departments	
- departmentId: Mã định danh duy nhất của chuyên khoa. - departmentName: Tên của chuyên khoa. - departmentPhone: Số điện thoại liên lạc. - departmentEmail: Địa chỉ thư điện tử của khoa. - hospitalId: ID của bệnh viện mà chuyên khoa này trực thuộc.	

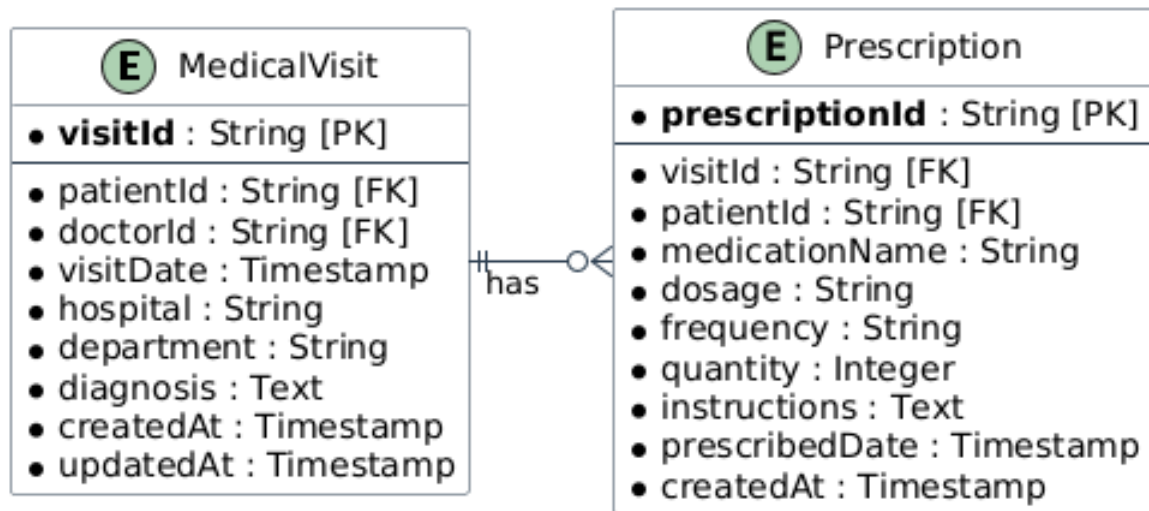
Hình 6. Bảng department

Các quy tắc nghiệp vụ của bảng Departments: Để dữ liệu trong bảng Departments tồn tại thì bắt buộc trong bảng Hospital phải tồn tại dữ liệu có hospitalId tương ứng.

5.2.5 EHR service

EHR service bao gồm dữ liệu lấy từ bảng Patients thuộc patient-service và các dữ liệu riêng của EHR service là Prescriptions-Đơn thuốc và MedicalVisists-Lịch sử khám bệnh của bệnh nhân. Do đó, dữ liệu phần dữ liệu của bệnh nhân bắt buộc phải được đồng bộ với dữ liệu của bảng Patients. Các chức năng của service là thêm, sửa, xóa thường được thực hiện bởi bác sĩ.

Thiết kế dữ liệu cho bảng MedicalVisits:



Hình 7. Bảng MedicalVisit và Prescription

Bảng MedicalVisits	Bảng Prescriptions
- visitId: Mã định danh duy nhất cho lượt khám.	- prescriptionId: Mã định danh duy nhất của đơn thuốc.
- patientId: ID của bệnh nhân.	- visitId: ID của lượt khám bệnh.
- doctorId: ID của bác sĩ phụ trách.	- patientId: ID của bệnh nhân được kê đơn thuốc này.
- visitDate: Ngày và giờ cụ thể.	- medicationName: Tên loại thuốc.
- hospital: Id bệnh viện khám	- dosage: Liều lượng sử dụng.
- department: Id khoa khám	- frequency: Tần suất sử dụng.
- diagnosis: Nội dung chẩn đoán bệnh.	- quantity: Tổng số lượng thuốc.
- prescriptions: Danh sách các đơn thuốc được kê.	- instructions: Hướng dẫn chi tiết cách dùng.
- createdAt: Thời điểm hồ sơ lượt khám được tạo trên hệ thống.	- prescribedDate: Ngày giờ cụ thể đơn thuốc được kê.
- updatedAt: Thời điểm cập nhật thông tin lượt khám gần nhất.	- createdAt: Thời điểm bản ghi đơn thuốc được lưu vào cơ sở dữ liệu.

Các quy tắc nghiệp vụ của bảng MedicalVisits: để đảm bảo tính thống nhất của dữ liệu trong toàn bộ hệ thống, Dữ liệu của bảng MedicalVisits chỉ tồn tại khi dữ liệu trong các bảng patients, doctors, hospitals, departments có các đối tượng có id tương ứng với các trường patientId, doctorId, hospital, department.

Các quy tắc nghiệp vụ của bảng Prescriptions: tương tự dữ liệu trong bảng Prescriptions chỉ tồn tại khi tồn tại các đối tượng thuộc bảng patients và MedicalVisits có id tương ứng các trường: visitId, patientId.

5.2.6 Appointment service

Service này ghi nhận toàn bộ thông tin các cuộc hẹn được đặt trong hệ thống. Chức năng chính của service này là hỗ trợ người dùng đặt lịch hẹn và các thay đổi trên lịch hẹn(ví dụ: người dùng huỷ lịch, ...). Đối tượng tương tác chính trên cơ sở dữ liệu của service này là bác sĩ và bệnh nhân.

Thiết kế dữ liệu cho bảng Appointments như sau:

- appointmentId: Mã định danh duy nhất của lịch hẹn.
- doctorId / doctorName: ID của bác sĩ và tên bác sĩ.
- patientId / patientName: ID và tên của bệnh nhân đặt lịch.
- hospitalId / hospitalName: ID và tên bệnh viện nơi diễn ra cuộc hẹn.
- departmentId / departmentName: ID và tên chuyên khoa.
- timeSlotId: ID liên kết với khung giờ cụ thể trong lịch làm việc của bác sĩ.
- appointmentDateTime: Ngày và giờ chính xác của lịch hẹn khám.
- status: Trạng thái hiện tại của lịch hẹn
- pendingCreatedAt: Thời điểm lịch hẹn rơi vào trạng thái PENDING.
- notes: Các ghi chú bổ sung của bệnh nhân khi đặt lịch.
- reason: Lý do khám bệnh hoặc triệu chứng sơ bộ mà bệnh nhân cung cấp.
- createdAt: Thời điểm bản ghi lịch hẹn được tạo.
- updatedAt: Thời điểm cập nhật thông tin lịch hẹn gần nhất.

E Appointment
• appointmentId : String [PK] • doctorId : String • doctorName : String • patientId : String • patientName : String • hospitalId : String • hospitalName : String • departmentId : String • departmentName : String • timeSlotId : String • appointmentDateTime : LocalDateTime • status : String • pendingCreatedAt : LocalDateTime • notes : String • reason : String • createdAt : LocalDateTime • updatedAt : LocalDateTime

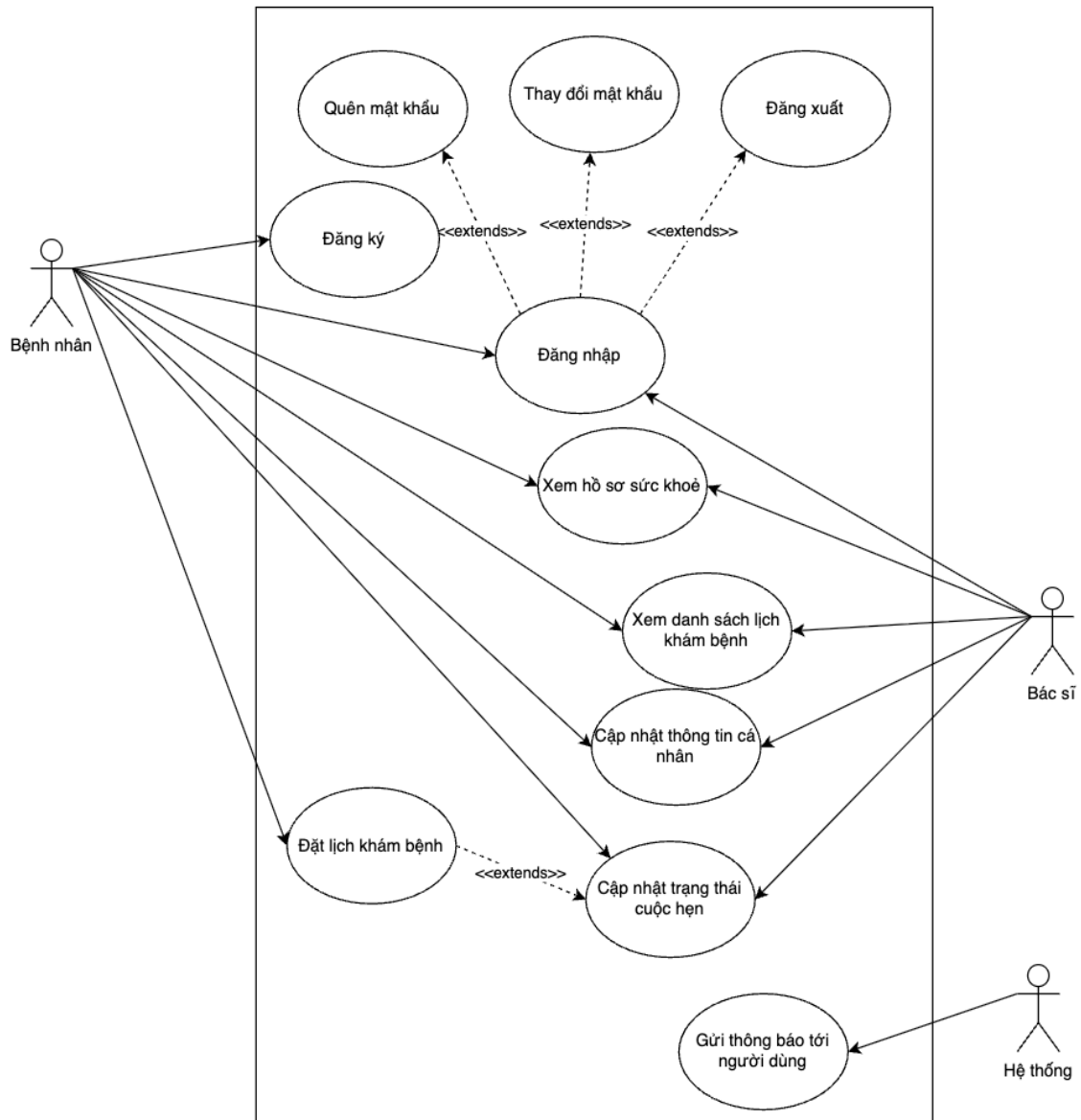
Hình 8. Bảng Appointment

Các quy tắc nghiệp vụ:

- Các trường dữ liệu `doctorId`, `doctorName`, `patientId`, `patientName`, `hospitalId`, `hospitalName`, `departmentId`, `departmentName`, `timeSlotId` là các trường dữ liệu tham chiếu đến các bảng `doctors`, `patients`, `hospitals`, `departments`, `timeslots`. Do đó thể thống nhất dữ liệu phải đảm bảo đã tồn tại các đối tượng có dữ liệu tương trong các bảng ở các service.
- `appointmentDateTime`: bắt buộc phải khớp với dữ liệu ngày hẹn trong bảng `TimeSlots` trong `doctor-service`.
- `status`: dữ liệu trong trường `status` phải thuộc danh sách sau: “PENDING”, “CONFIRMED”, “CANCELLED”, “COMPLETED”, “NO_SHOW”.

CHƯƠNG 6. SƠ ĐỒ THIẾT KẾ CHỨC NĂNG

6.1 Sơ đồ Use-case



Hình 9. Sơ đồ use case tổng quát

6.1.1 Đăng ký

Thuộc tính	Mô tả
Use Case ID	UC-001

Tên Use Case	Đăng ký
Actors	Bệnh nhân
Mô tả	Tạo tài khoản nếu người dùng chưa có tài khoản trên hệ thống.
Tiền điều kiện	Các thông tin sau chưa xuất hiện trong hệ thống với vai trò bệnh nhân: - Số căn cước công dân - Số điện thoại - Email
Hậu điều kiện	Tài khoản bệnh nhân thành công và lưu dữ liệu vào cơ sở dữ liệu của các service liên quan.
Luồng chính	1. Người dùng chọn tạo tài khoản 2. Người dùng nhập các thông tin cần thiết 3. Người dùng chọn đăng ký 4. Hệ thống gọi đến server để đăng ký tài khoản 5. Tạo tài khoản thành công chuyển người dùng về trang đăng nhập và yêu cầu người dùng đăng nhập bằng tài khoản vừa tạo. Hệ thống tự động gửi một email chào mừng.
Luồng thay thế	1a. Người dùng quay lại đăng ký.
Luồng ngoại lệ	4a. Dữ liệu không hợp lệ - Nếu tài khoản đã tồn tại hệ thống báo lỗi “Tài khoản đã tồn tại”. - Nếu dữ liệu không hợp lệ hệ thống báo “Dữ liệu chưa hợp lệ”.
Quy tắc nghiệp vụ	- Các trường dữ liệu email, phone, số căn cước công dân phải không được trùng. - Mật khẩu phải được bấm ra trước khi lưu vào cơ sở dữ liệu.

-
- Tạo user sau đó thực hiện tạo patient thông qua api đã cung cấp.
-

6.1.2 Đăng nhập

Thuộc tính	Mô tả
Use Case ID	UC-002
Tên Use Case	Đăng nhập
Actors	Bệnh nhân, bác sĩ và quản trị viên.
Mô tả	Cung cấp các phương thức đăng nhập vào hệ thống.
Tiền điều kiện	Người dùng đã có tài khoản với vai trò tương ứng.
Hậu điều kiện	Đăng nhập thành công vào hệ thống và được phép sử dụng các chức năng với vai trò của mình.
Luồng chính	1. Tại giao diện ngay khi vào ứng dụng. Người dùng nhập thông tin tên tài khoản và mật khẩu. Tên tài khoản có thể là email hoặc số điện thoại hoặc số căn cước công dân khi người dùng thực hiện đăng ký. 2. Hệ thống gọi tới server xác thực người dùng. 3. Xác thực thành công chuyển người dùng vào trang chính với vai trò tương ứng khi đăng nhập.
Luồng thay thế	1a. Người dùng chọn quên mật khẩu: chuyển người dùng đến trang quên mật khẩu. 1b. Người dùng chọn đăng ký: chuyển người dùng đến trang đăng ký.
Luồng ngoại lệ	2a. Dữ liệu không hợp lệ: Thông báo đăng nhập không thành công: “Tài khoản hoặc mật khẩu không chính xác”.
Quy tắc nghiệp vụ	- Cho phép đăng nhập với tên tài khoản là: email hoặc số điện thoại hoặc số căn cước công dân. - Khi nhập mật khẩu phải ẩn đi.

6.1.3 Quên mật khẩu

Thuộc tính	Mô tả
Use Case ID	UC-003
Tên Use Case	Quên mật khẩu
Actors	Bệnh nhân, bác sĩ và quản trị viên.
Mô tả	Cho phép người dùng khôi phục mật khẩu khi mất.
Tiền điều kiện	Người dùng đã có tài khoản trên hệ thống.
Hậu điều kiện	Cập nhật mật khẩu mới thành công.
Luồng chính	1. Người dùng chọn quên mật khẩu, hệ thống chuyển người dùng đến trang quên mật khẩu. 2. Người dùng nhập email đã đăng ký và hệ thống gửi một mã OTP đến email này. 3. Người dùng kiểm tra email và nhập mã OTP vừa nhận được. 4. Hệ thống kiểm tra mã OTP và chuyển người dùng đến trang cập nhật mật khẩu mới. 5. Người dùng cập nhật mật khẩu mới. 6. Hệ thống gọi tới server để cập nhật mật khẩu vào cơ sở dữ liệu. Thành công chuyển người dùng đến trang đăng nhập.
Luồng thay thế	1a. Người dùng thay đổi ý định chọn đăng nhập: Chuyển người dùng đến trang đăng nhập.
Luồng ngoại lệ	2a. Email người dùng nhập không hợp lệ: Thông báo lỗi "email không tồn tại" và yêu cầu người dùng nhập lại. 4a. Người dùng nhập mã OTP không chính xác. Hệ thống thông báo mã OTP không chính xác và giới hạn số lần nhập lại.
Quy tắc nghiệp vụ	- Mã OTP chỉ có giới hạn thời gian trong 5 phút.

-
- Mật khẩu khi cập nhật vào cơ sở dữ liệu phải được đưa qua hàm băm.
-

6.1.4 Thay đổi mật khẩu

Thuộc tính	Mô tả
Use Case ID	UC-004
Tên Use Case	Thay đổi mật khẩu
Actors	Bệnh nhân, bác sĩ và quản trị viên
Mô tả	Cho phép người dùng thay đổi mật khẩu nếu không cảm thấy an toàn.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Mật khẩu mới được cập nhật vào cơ sở dữ liệu.
Luồng chính	1. Người dùng chọn trang thông tin cá nhân và chọn thay đổi mật khẩu. 2. Hệ thống chuyển đến trang thay đổi mật khẩu. 3. Người dùng nhập thông tin cần thiết bao gồm: Mật khẩu hiện tại, mật khẩu mới và xác nhận mật khẩu mới. 4. Hệ thống gọi tới server thực hiện xác thực lại người dùng lần nữa và cập nhật mật khẩu.
Luồng thay thế	Người dùng thay đổi ý định: chuyển về trang xem thông tin cá nhân.
Luồng ngoại lệ	4a. Mật khẩu mới và xác nhận mật khẩu mới không khớp: Thông báo lỗi và yêu cầu người dùng nhập lại. 4b. Mật khẩu hiện tại không chính xác: Thông báo mật khẩu hiện tại không chính xác và yêu cầu người dùng nhập lại.
Quy tắc nghiệp vụ	- Mật khẩu mới và xác nhận mật khẩu mới phải trùng nhau. - Mật khẩu trước khi được cập nhật vào cơ sở dữ liệu phải đảm bảo được đi qua hàm băm.

6.1.5 Đăng xuất

Thuộc tính	Mô tả
Use Case ID	UC-005
Tên Use Case	Đăng xuất
Actors	Bệnh nhân, bác sĩ và quản trị viên.
Mô tả	Cho phép người dùng thoát khỏi hệ thống.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Người dùng thoát khỏi hệ thống.
Luồng chính	1. Người dùng chọn trang thông tin cá nhân, sau đó chọn đăng xuất. 2. Hệ thống xóa các thông tin đăng nhập đã lưu và chuyển người dùng ra trang đăng nhập.
Luồng thay thế	1a. Người dùng chọn đổi mật khẩu: hệ thống chuyển người dùng đến chức năng đổi mật khẩu.
Quy tắc nghiệp vụ	Thông tin đăng nhập của người dùng lưu trên local storage phải được xóa sạch.

6.1.6 Xem hồ sơ sức khỏe

Thuộc tính	Mô tả
Use Case ID	UC-006
Tên Use Case	Xem hồ sơ sức khỏe
Actors	Bệnh nhân và bác sĩ.
Mô tả	Cho phép người dùng xem thông tin hồ sơ sức khỏe EHR.
Tiền điều kiện	Người dùng đã đăng nhập.

Hậu điều kiện	Thông tin hồ sơ sức khỏe được hiển thị lên giao diện bao gồm: Thông tin bệnh nhân, thông tin lịch sử khám bệnh và đơn thuốc.
Luồng chính	1. Người dùng chọn xem hồ sơ sức khỏe. 2. Hệ thống gọi tới server để lấy thông tin và hiển thị lên giao diện.
Luồng ngoại lệ	2a. Hệ thống không tải được dữ liệu. Báo lỗi và yêu cầu người dùng truy cập lại vào trang.

6.1.7 Xem danh sách lịch khám bệnh

Thuộc tính	Mô tả
Use Case ID	UC-007
Tên Use Case	Xem lịch sử lịch khám
Actors	Bệnh nhân và bác sĩ.
Mô tả	Cho phép bệnh nhân xem thông tin lịch hẹn mình đã đặt và cho phép bác sĩ xem lịch làm việc của mình.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	- Hiển thị toàn bộ lịch hẹn của của bệnh nhân đã đặt theo thứ tự từ hiện tại đến quá khứ. - Đối với bác sĩ phải hiển thị lịch làm việc theo ngày và theo thứ tự giờ.
Luồng chính	1. Bệnh nhân chọn xem lịch sử đặt lịch hẹn. Bác sĩ chọn thông tin lịch làm việc. 2. Hệ thống gọi tới server lấy thông tin lịch làm việc.
Luồng thay thế	1. Bệnh nhân chọn huỷ lịch 2. Bác sĩ chọn huỷ lịch hoặc xác nhận hoàn thành
Luồng ngoại lệ	2a. Hệ thống không tải được lịch hẹn: yêu cầu tải lại trang.

Quy tắc nghiệp vụ	- Hiện thị thông tin lịch hẹn với bệnh nhân: Tên bác sĩ, tên bệnh viện, tên khoa, giờ khám, trạng thái, nút huỷ lịch chỉ hiện thị trước 24 giờ khi đến thời gian khám. - Hiện thị thông tin lịch hẹn với bác sĩ: Tên bệnh nhân, tên bác sĩ, tên khoa, trạng thái. Nút huỷ lịch và nút hoàn thành lịch.
-------------------	---

6.1.8 Cập nhật thông tin cá nhân

Thuộc tính	Mô tả
Use Case ID	UC-008
Tên Use Case	Cập nhật thông tin cá nhân
Actors	Bệnh nhân và bác sĩ.
Mô tả	Cho phép người dùng xem và cập nhật thông tin cá nhân
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Thông tin cá nhân được cập nhật trong database. Thông báo xác nhận được hiển thị.
Luồng chính	1. Người dùng chọn quản lý hồ sơ, sau đó chọn biểu tượng cập nhật thông tin. 2. Người dùng cập nhật những thông tin được cho phép. 3. Hệ thống kiểm tra về cập nhật vào cơ sở dữ liệu.
Luồng thay thế	1a. Người dùng chọn đổi mật khẩu: chuyển đến trang đổi mật khẩu. 1b. Người dùng chọn đăng xuất: đăng xuất người dùng. 1c. Người dùng chọn thay đổi mật khẩu: chuyển người dùng đến trang thay đổi mật khẩu.
Luồng ngoại lệ	Hệ thống xác thực thông tin chưa hợp lí: Thông tin nào chưa hợp lí sẽ được thông báo và người dùng sẽ nhập lại.

Quy tắc nghiệp vụ	- Chú ý các thông tin có thể thay đổi được liệt kê trong nghiệp vụ, đặc biệt là các trường thông tin duy nhất trong hệ thống.
-------------------	---

6.1.9 Đặt lịch khám bệnh

Thuộc tính	Mô tả
Use Case ID	UC-009
Tên Use Case	Đặt lịch khám bệnh
Actors	Bệnh nhân
Mô tả	Cho phép người dùng đặt lịch khám bệnh tại cơ sở khám bệnh và chọn bác sĩ.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Thông tin lịch khám bệnh được ghi nhận và xác nhận lịch hẹn có được “duyet” hay “hủy”.
Luồng chính	1. Bệnh nhân chọn đặt lịch hẹn, hệ thống chuyển đến trang đặt lịch hẹn 2. Bệnh nhân chọn bệnh viên, chọn khoa, chọn bác sĩ và chọn thời gian còn trống trong danh sách và nhấn đặt lịch. 3. Hệ thống kiểm tra thông tin sau đó tạo lịch hẹn và kiểm tra trạng thái. 4. Nếu đặt lịch thành công gửi email thông báo đến bệnh nhân và bác sĩ.
Luồng ngoại lệ	- Hệ thống đặt lịch hẹn gặp lỗi: hệ thống thông báo đặt lịch hẹn không thành công. - Trùng lịch hẹn: Hệ thống thông báo đặt lịch hẹn không thành công và lịch hẹn sẽ được hủy
Quy tắc nghiệp vụ	Thông tin bác sĩ, bệnh viện, khoa phải là những thông tin đã tồn tại.

6.1.10 Cập nhật trạng thái lịch hẹn

Thuộc tính	Mô tả
Use Case ID	UC-0010
Tên Use Case	Cập nhật trạng thái lịch hẹn
Actors	Bệnh nhân và bác sĩ.
Mô tả	Cho phép bệnh nhân huỷ lịch hoặc bác sĩ huỷ lịch và xác nhận hoàn thành.
Tiền điều kiện	Người dùng đã đăng nhập. Lịch hẹn đã được đặt và ở trạng thái “CONFIRMED”.
Hậu điều kiện	Thông tin trạng thái mới của lịch hẹn được cập nhật.
Luồng chính	1. Bệnh nhân truy cập vào lịch sử đặt lịch hẹn và bác sĩ truy cập vào thông tin lịch làm việc. 2. Bệnh nhân được phép huỷ lịch, bác sĩ được phép huỷ lịch hoặc xác nhận hoàn thành lịch. 3. Hệ thống gọi tới server thực hiện cập nhật theo yêu cầu. 4. Nếu thành công cập nhật giao diện ngay lập tức.
Luồng ngoại lệ	Cập nhật không thành công: hệ thống báo lỗi và yêu cầu thực hiện lại.
Quy tắc nghiệp vụ	- Lịch hẹn chỉ được chuyển từ “CONFIRMED” sang “CANCELLED” hoặc “COMPLETED”. - Bệnh nhân chỉ được huỷ lịch. Bác sĩ được huỷ hoặc xác nhận hoàn thành.

6.1.11 Gửi thông báo tới người dùng

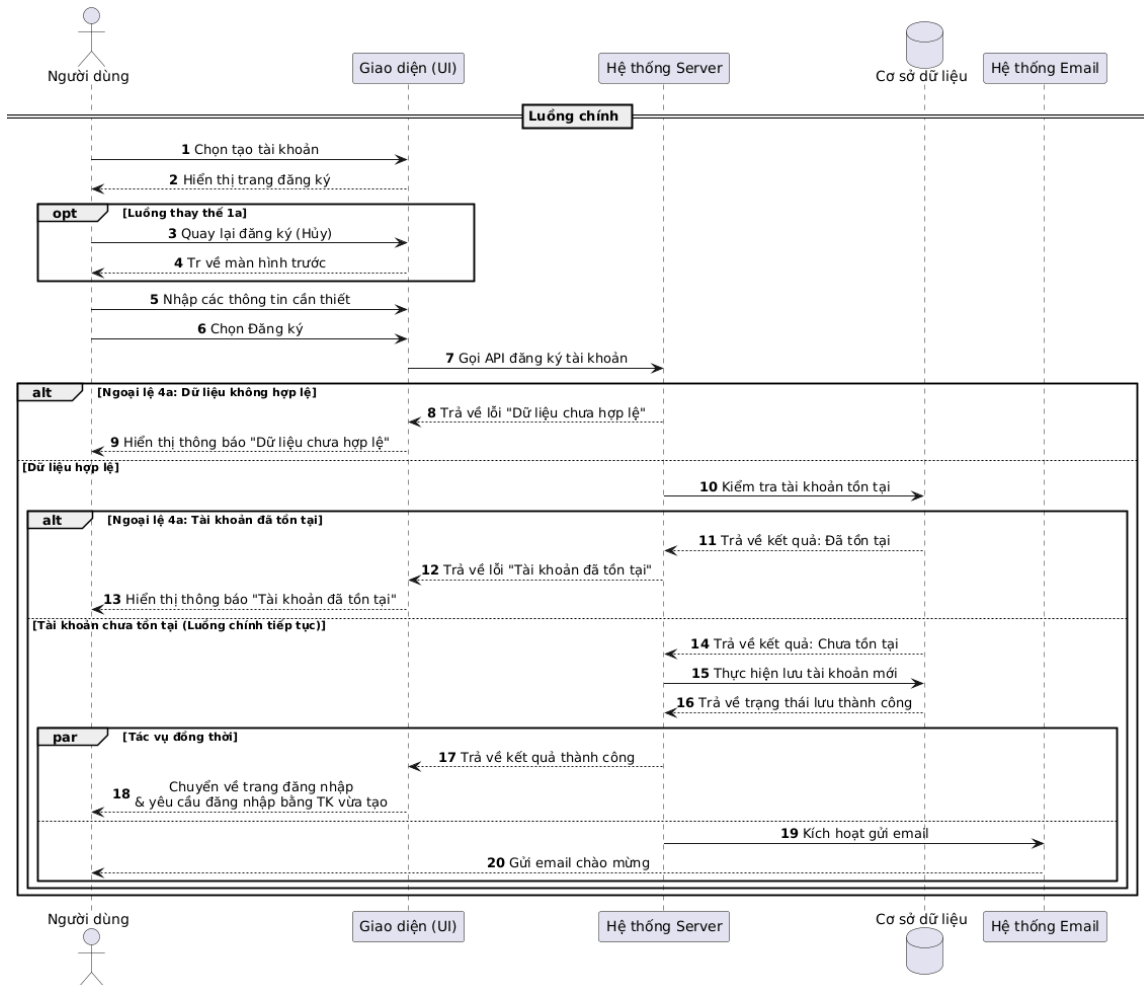
Thuộc tính	Mô tả
Use Case ID	UC-0011

Tên Use Case	Gửi thông báo thông qua email
Actors	Hệ thống
Mô tả	Hệ thống tự động gửi các thông báo qua email khi người dùng: tạo tài khoản thành công, đặt lịch hẹn thành công, quên mật khẩu.
Tiền điều kiện	Người dùng có tài khoản hợp lệ.
Hậu điều kiện	Thông báo được gửi tới người dùng.
Luồng chính	Hệ thống thực hiện gửi email bằng các thông tin đã lưu của người dùng.
Luồng ngoại lệ	Hệ thống gửi email bị lỗi. Người dùng xem các thông tin này trực tiếp trong hệ thống.
Quy tắc nghiệp vụ	Hệ thống chỉ gửi tới người dùng các thông tin cần thiết và thực hiện bảo mật thông tin.

6.2 Sơ đồ tuần tự

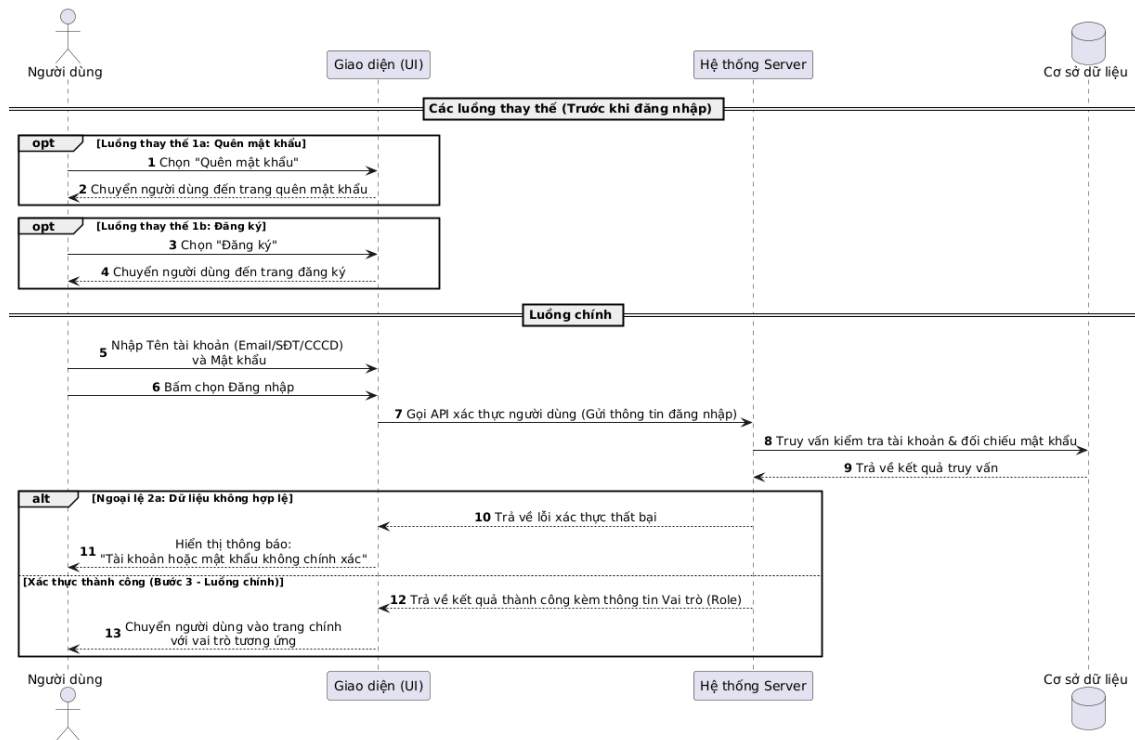
Sơ đồ tuần tự trực quan hoá quy trình thực hiện một chức năng đi từ người dùng đến cơ sở dữ liệu và gọi tới service khác (nếu có).

6.2.1 Đăng ký



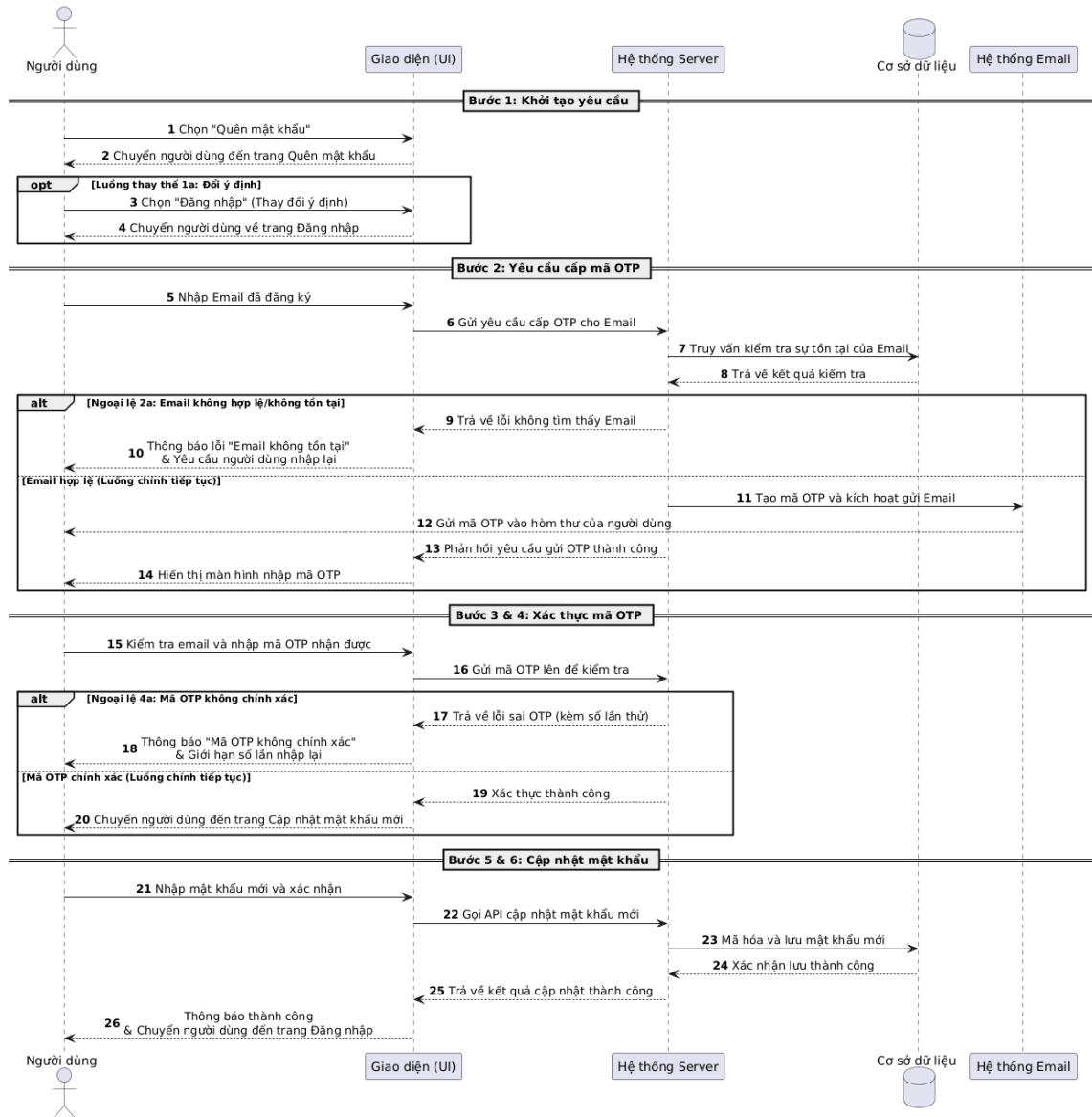
Hình 10. Sơ đồ tuần tự chức năng đăng ký

6.2.2 Đăng nhập



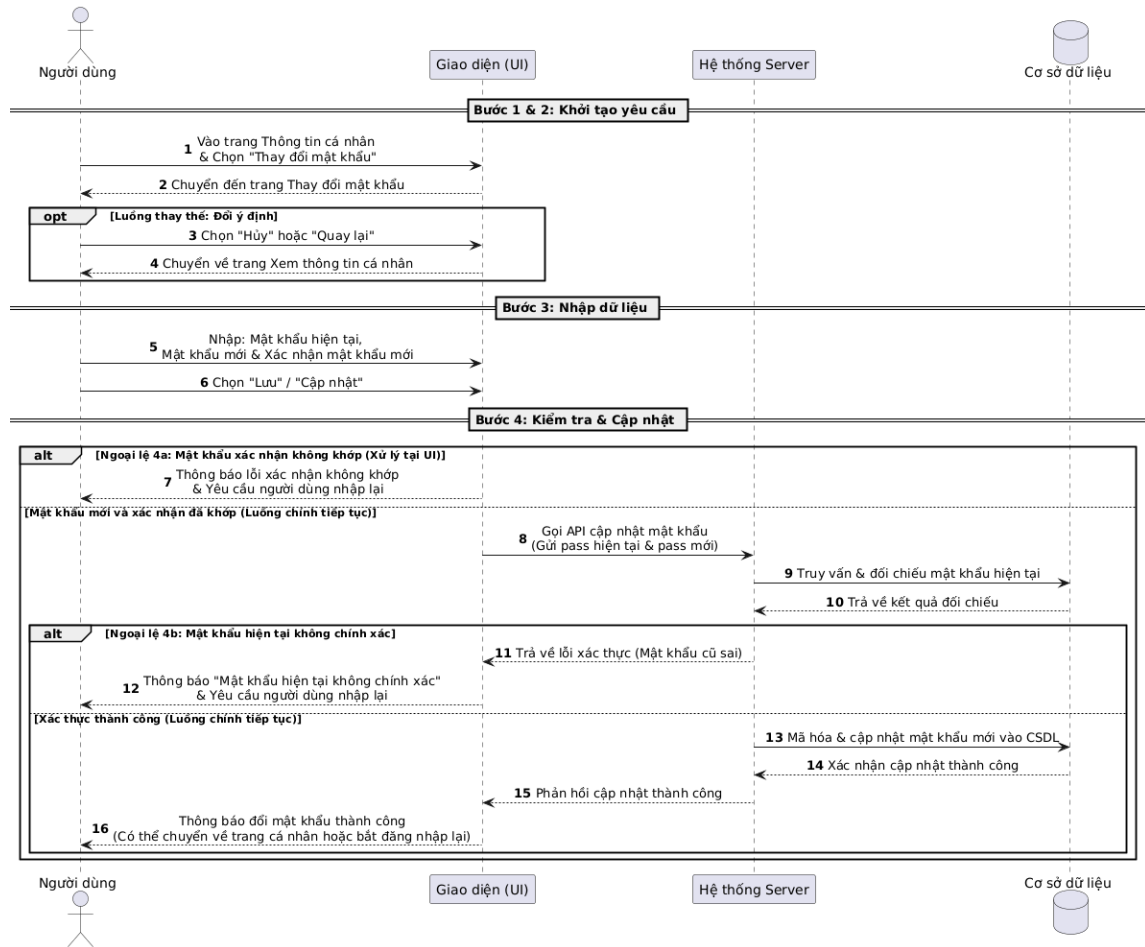
Hình 11. Sơ đồ tuần tự chức năng đăng nhập

6.2.3 Quên mật khẩu



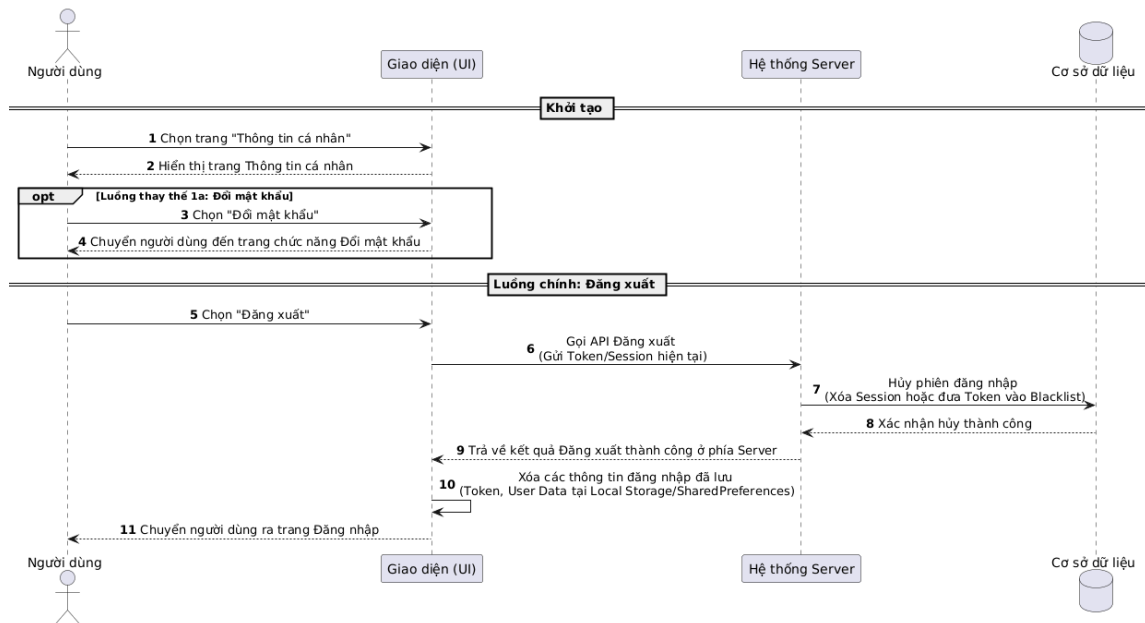
Hình 12. Sơ đồ tuần tự chức năng quên mật khẩu

6.2.4 Thay đổi mật khẩu



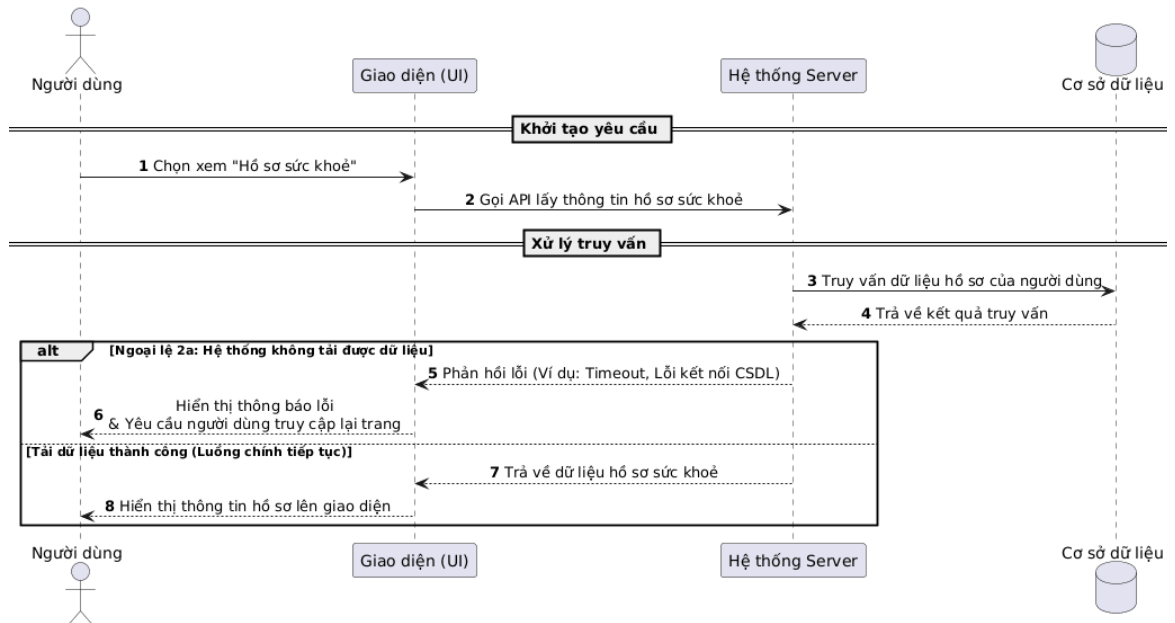
Hình 13. Sơ đồ tuần tự chức năng quên mật khẩu

6.2.5 Đăng xuất



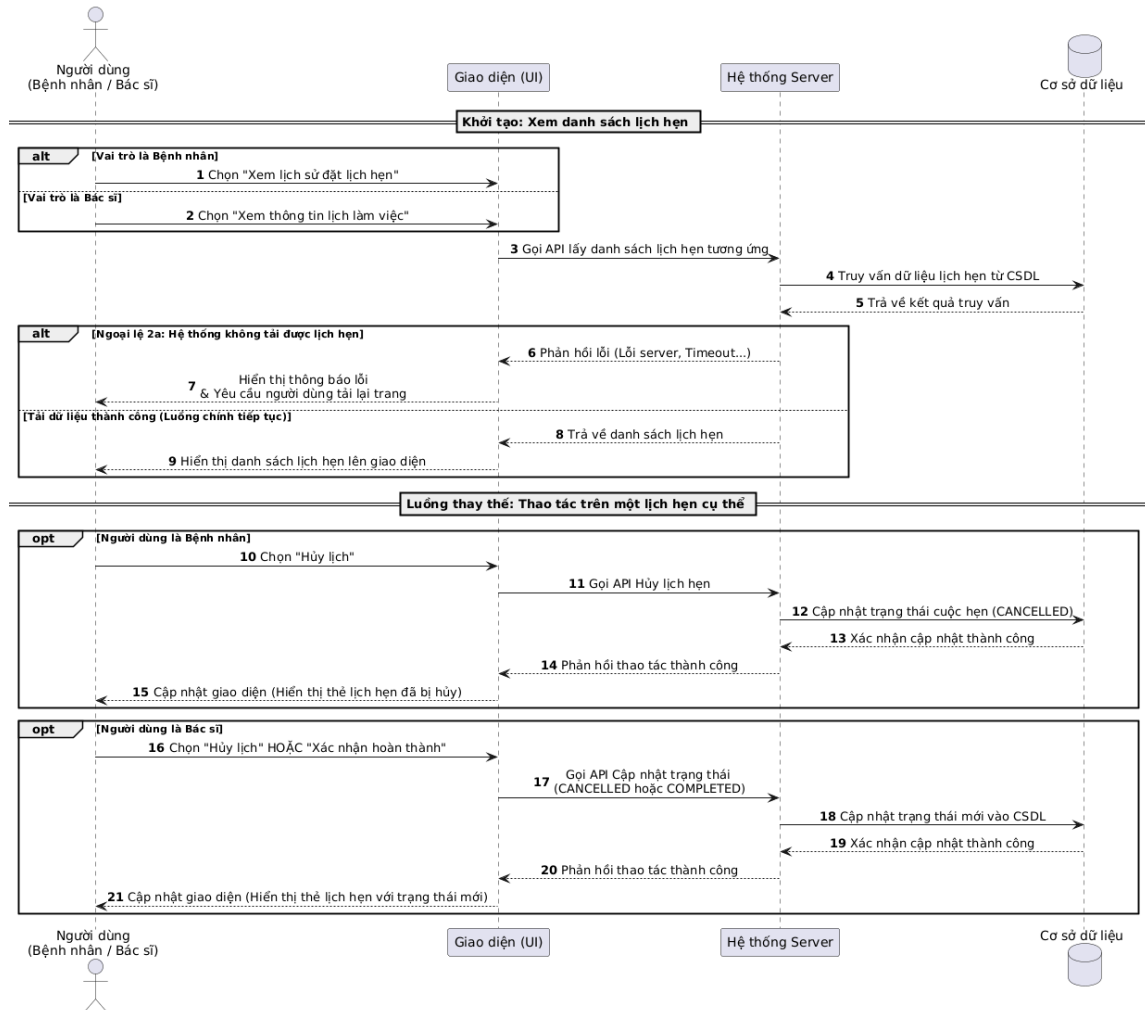
Hình 14. Sơ đồ tuần tự chức năng đăng xuất

6.2.6 Xem hồ sơ sức khỏe



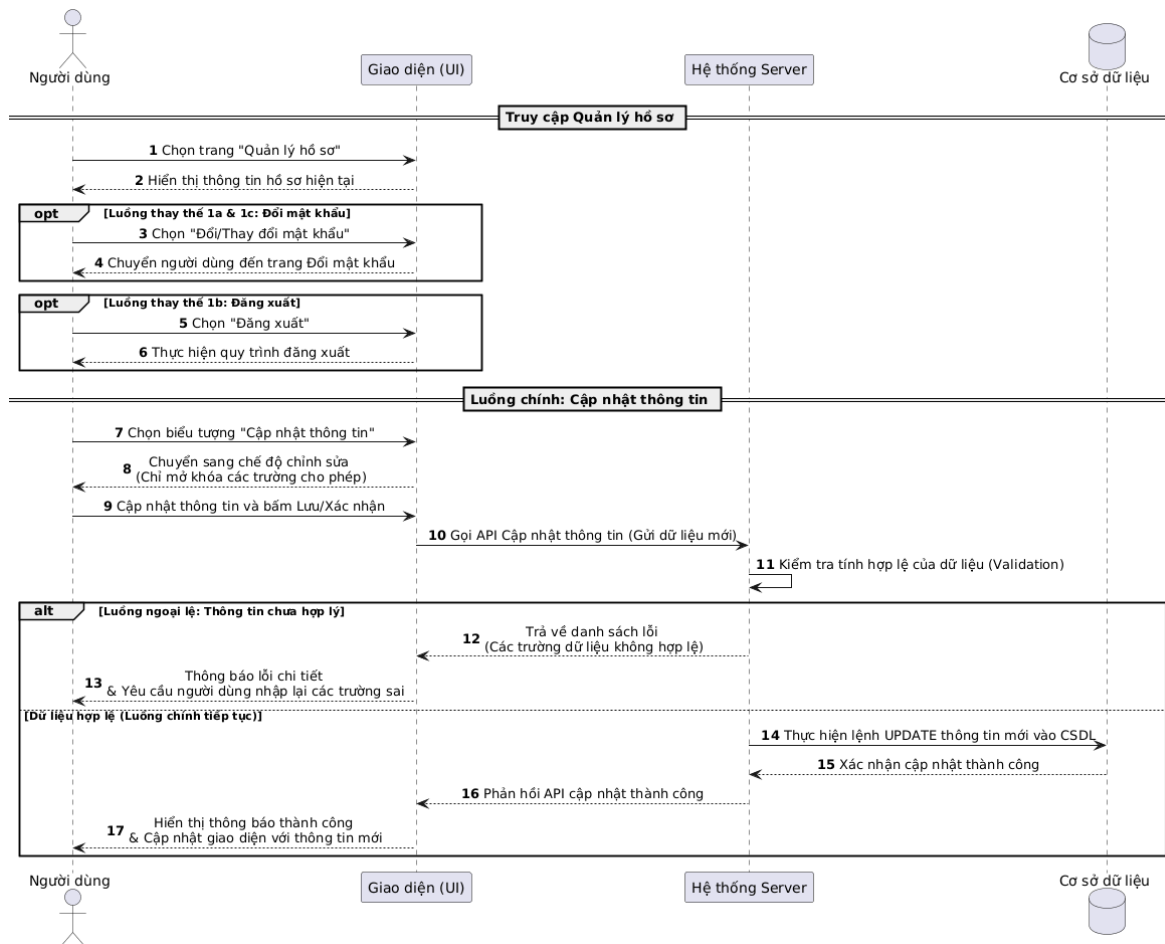
Hình 15. Sơ đồ tuần tự chức năng xem hồ sơ

6.2.7 Xem danh sách lịch khám bệnh



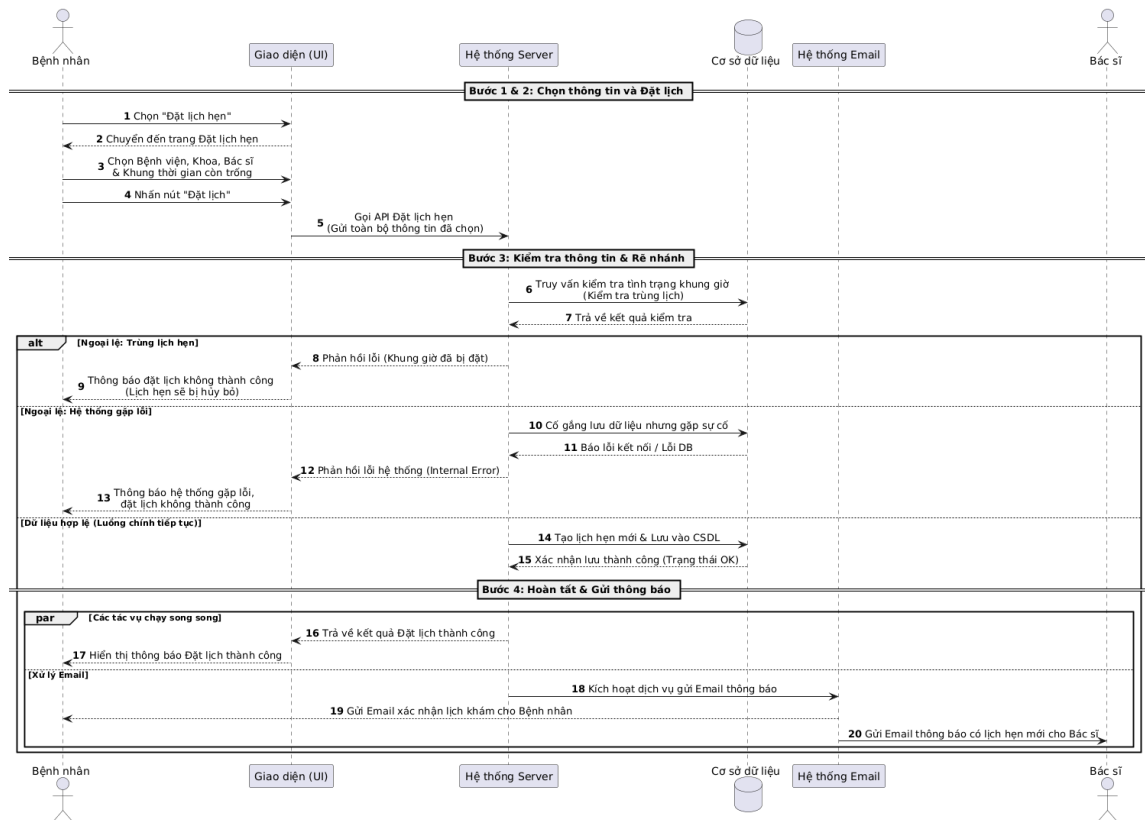
Hình 16. Sơ đồ tuần tự chức năng xem lịch khám bệnh

6.2.8 Cập nhật thông tin cá nhân



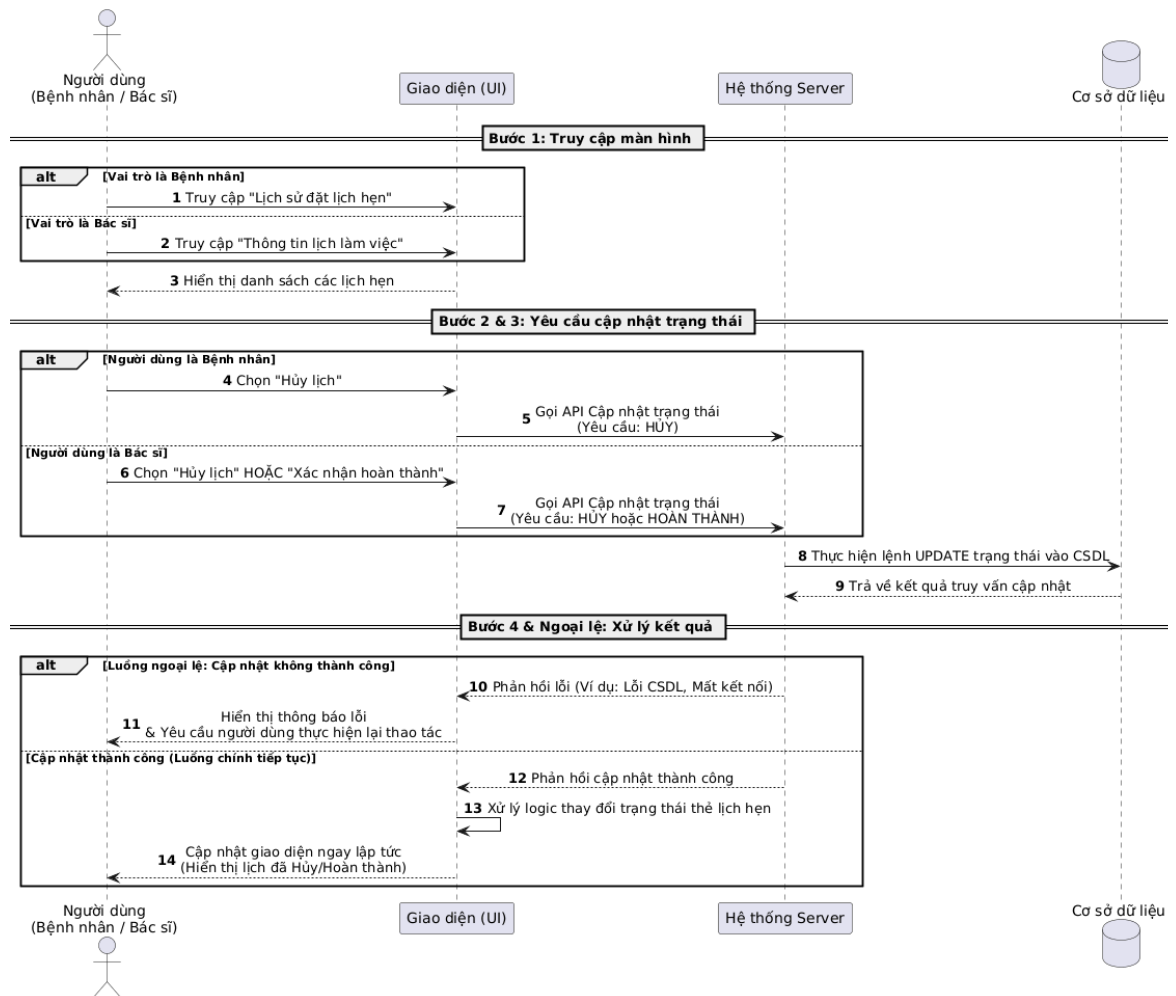
Hình 17. Sơ đồ tuần tự chức năng cập nhật thông tin

6.2.9 Đặt lịch khám bệnh



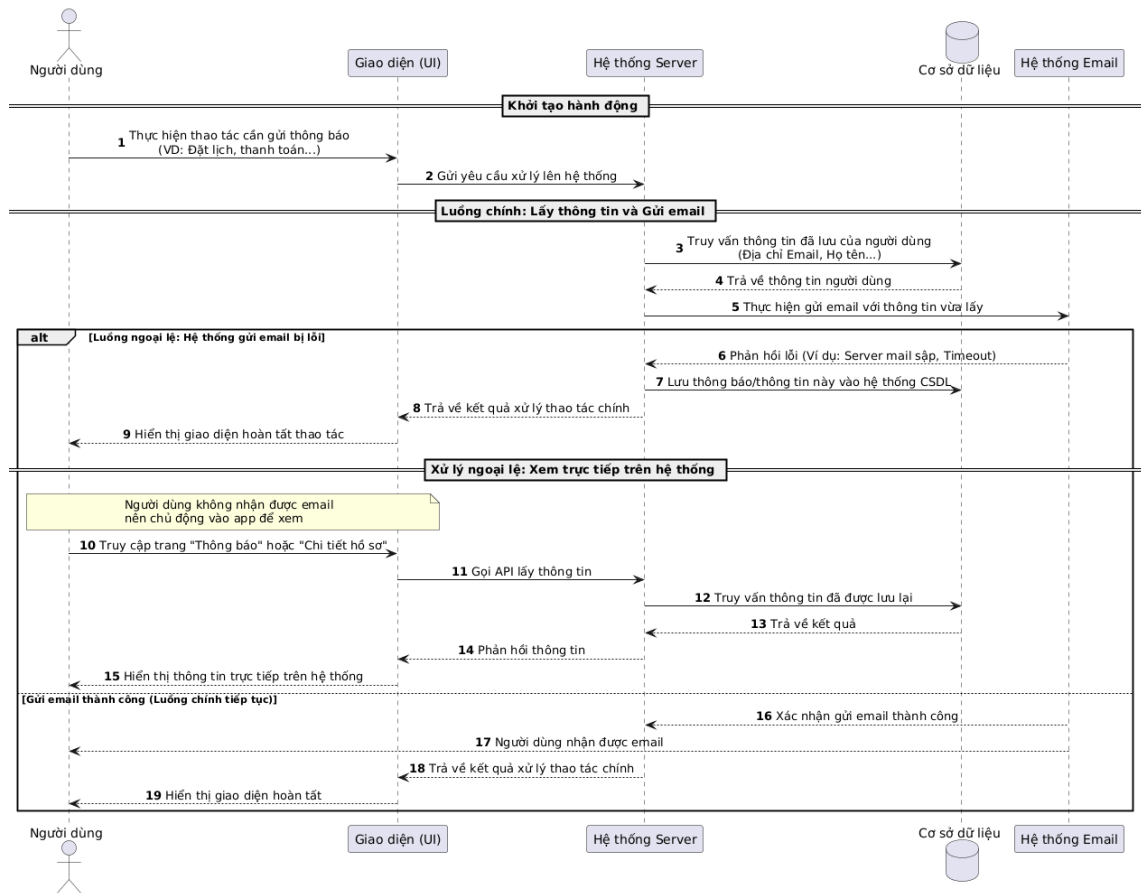
Hình 18. Sơ đồ tuần tự chức năng đặt lịch khám bệnh

6.2.10 Cập nhật trạng thái lịch hẹn



Hình 19. Sơ đồ tuần tự chức năng cập nhật trạng thái lịch hẹn

6.2.11 Gửi thông báo tới người dùng



Hình 20. Sơ đồ tuần tự chức năng gửi thông báo

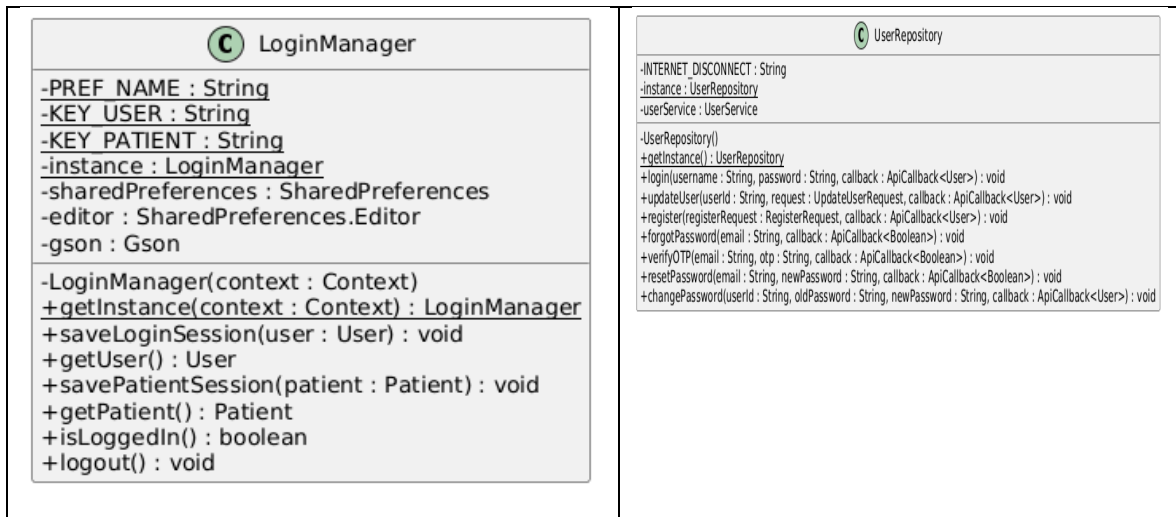
CHƯƠNG 7. ÁP DỤNG PATTERN

7.1 Singleton Pattern

Vị trí áp dụng: Tại giao diện người dùng (front-end)

Lý do áp dụng: Áp dụng để lưu thông tin của người dùng đang thực hiện đăng nhập vào hệ thống, vì tại mỗi thời điểm thì chỉ có 1 tài khoản được phép đăng nhập vào giao diện người dùng. Áp dụng tạo đối tượng gọi API nhằm giảm việc hao phí tài nguyên.

Sơ đồ lớp:



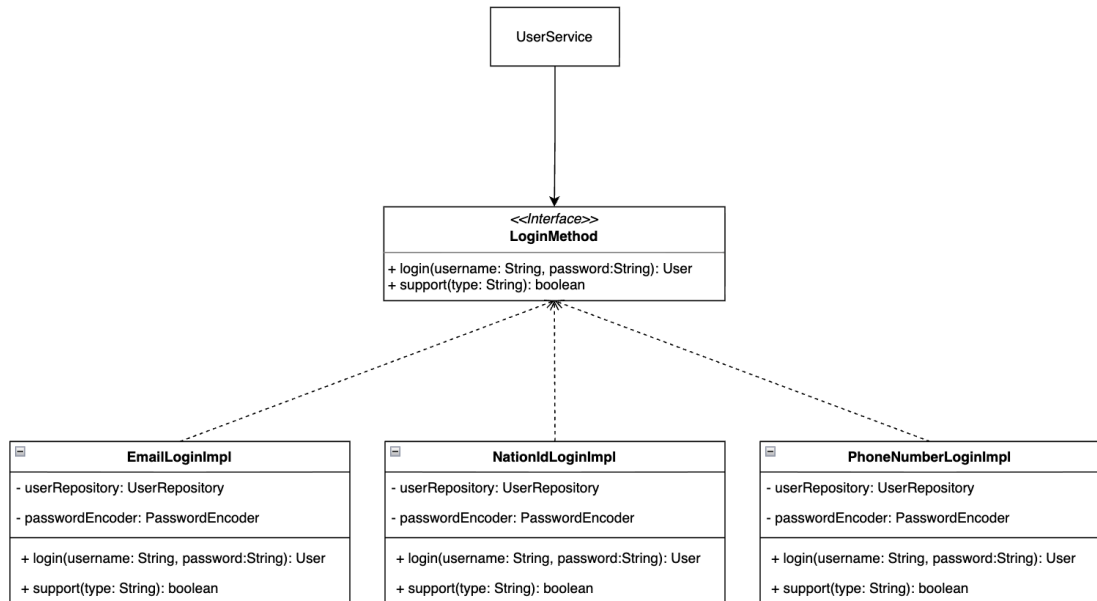
Hình 21. Sơ đồ lớp Singleton Pattern

7.2 Strategy Pattern

Vị trí áp dụng: áp dụng tại backend cho chức năng đăng nhập(hỗ trợ đăng nhập qua 3 cách là email, số điện thoại, số căn cước công dân và mật khẩu).

Lý do áp dụng:Nhận thấy trong hệ thống các trường thông tin email, số điện thoại và căn cước công dân đều là các trường không được trùng lặp của mỗi tài khoản, do đó áp dụng strategy pattern giúp mở rộng chức năng đăng nhập, giúp đăng nhập bằng nhiều cách hơn.

Sơ đồ lớp:



Hình 22. Sơ đồ lớp Strategy Pattern

Cách mở rộng: Khi có một phương thức đăng nhập mới chỉ cần xây dựng lớp cho phương thức đó và hiện thực interface `LoginMethod`, sau đó hiện thực phương thức `login` và `gán kiểu`.

7.3 Template Method Paterrn

Vị trí áp dụng: Tại phần chuyển đổi trạng thái cuộc hẹn ở service Đặt lịch (`AppointmentService`)

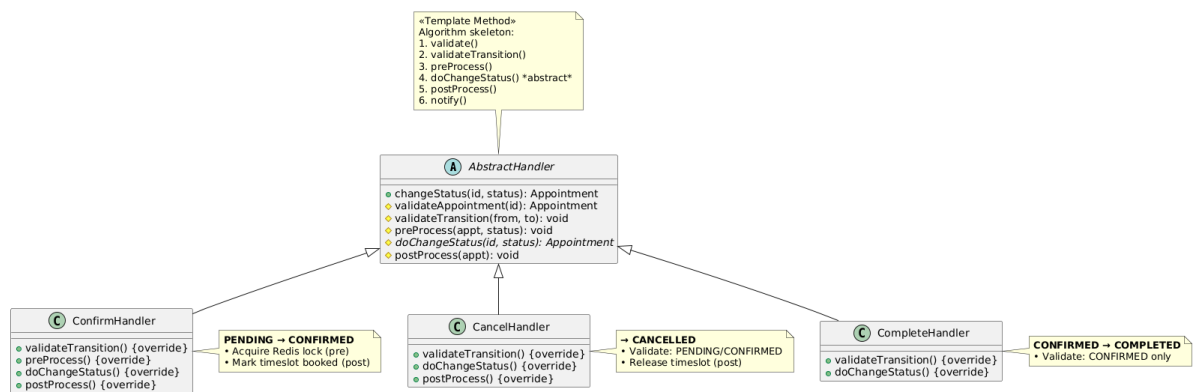
Lý do áp dụng:

Đảm bảo consistency cho toàn bộ status transition workflow
Mọi thay đổi status (CONFIRM, CANCEL, COMPLETE) đều phải thực hiện theo cùng một quy trình 6 bước:

- Validate appointment tồn tại
- Validate transition hợp lệ
- Pre-processing (acquire lock, check conditions)
- Thực hiện đổi status (abstract)
- Post-processing (release lock, notify)

- Thông báo service liên quan
- Loại bỏ code duplication: Các bước chung (validate, logging) được implement 1 lần trong abstract class, tránh lặp lại trong mỗi concrete handler.
- Control flow bị lock lại: Template method changeStatus() được khai báo final → không thể subclass override → đảm bảo thuật toán không bị thay đổi sai.
- Dễ mở rộng, khó phá hoại: Thêm status mới chỉ cần tạo class mới, không sửa code cũ. Template method đảm bảo các bước bắt buộc luôn được thực thi.

Sơ đồ lớp:



Hình 23. Sơ đồ lớp Template Method Pattern

Cách mở rộng:

Khi cần thêm status mới (ví dụ: NO_SHOW - bệnh nhân không đến khám):

Thêm constant (NO_SHOW) vào enum sau đó tạo concrete class mới (NoShowHandler) kế thừa abstract class (AbstractAppointmentStatusHandler).

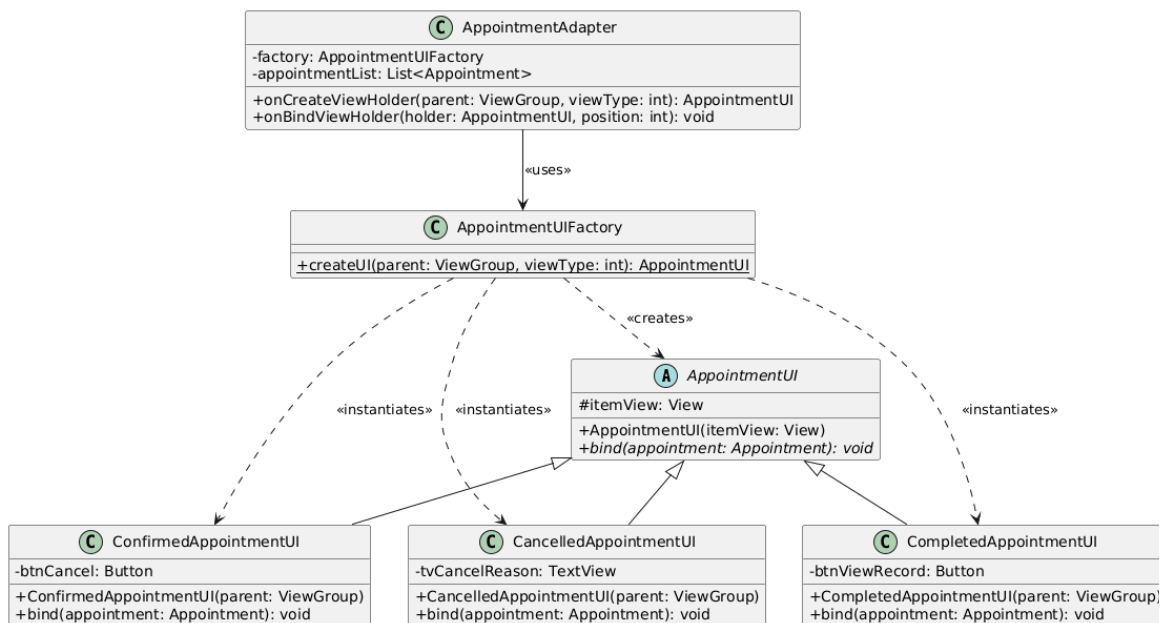
7.4 Simple Factory Pattern

Vị trí áp dụng: Pattern được áp dụng tại giao diện ứng dụng di động, cụ thể là bên trong lớp AppointmentAdapter thuộc tính năng Xem lịch sử cuộc hẹn. Nó chịu trách nhiệm khởi tạo các giao diện thẻ (CardView / ViewHolder) khác nhau dựa trên trạng thái hiện tại của cuộc hẹn, bao gồm: CONFIRMED (Đã xác nhận), COMPLETED (Đã hoàn thành), và CANCELLED (Đã hủy).

Lý do áp dụng:

- Giải quyết sự chồng chéo: Nếu không dùng Factory, các hàm `onCreateViewHolder` và `onBindViewHolder` của Adapter sẽ chứa hàng loạt câu lệnh `if-else` hoặc `switch-case` phức tạp để kiểm tra trạng thái và bơm giao diện tương ứng. Điều này vi phạm nguyên lý Trách nhiệm duy nhất làm lớp Adapter phình to và khó kiểm soát.
- Tách biệt logic khởi tạo: Simple Factory gom toàn bộ logic quyết định "sẽ tạo ra giao diện nào" vào một lớp duy nhất là `AppointmentUIFactory`. Adapter giờ đây chỉ đóng vai trò gọi Factory để lấy giao diện mà không cần quan tâm giao diện đó được khởi tạo phức tạp ra sao.
- Tăng tính thống nhất và tái sử dụng: Dữ liệu tiêm vào các giao diện được đồng bộ qua một phương thức chung `bind()`, giúp logic code mạch lạc và dễ dàng chỉnh sửa và mở rộng.

Sơ đồ lớp:



Hình 24. Sơ đồ lớp Simple Factory Pattern

Cách mở rộng: Nếu có trạng thái cuộc hẹn mới ta chỉ cần:

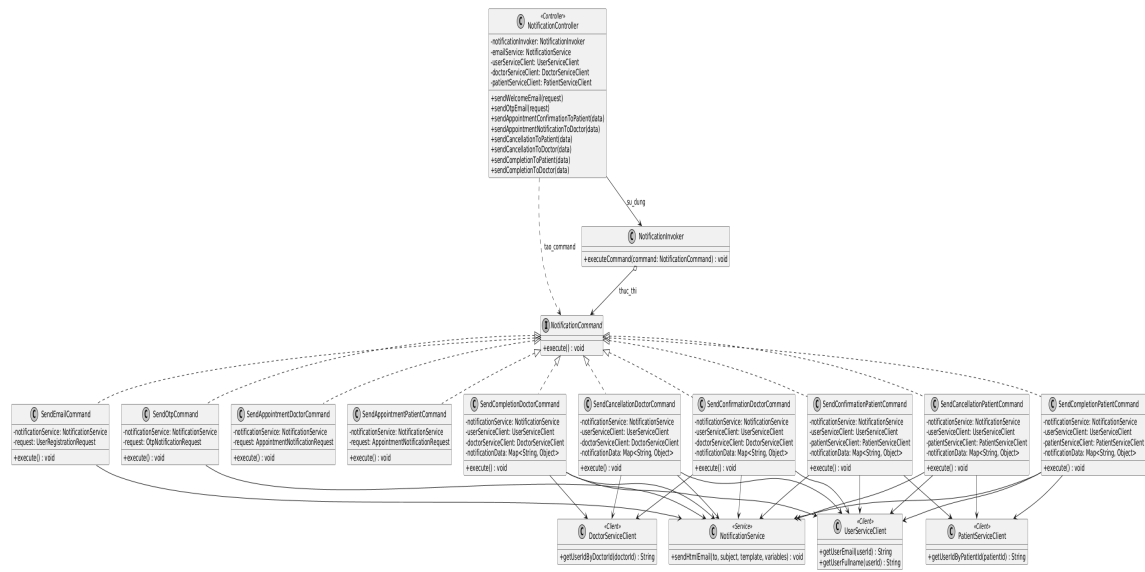
- Tạo thêm một lớp mới kế thừa từ lớp trừu tượng `AppointmentUI`, thiết kế layout và định nghĩa cách đồ dữ liệu tại hàm `bind()`.
- Bước 2: Bổ sung thêm một nhánh `case` cho đó vào bên trong lớp `AppointmentUIFactory` để hệ thống biết cách gọi lớp mới này.

7.5 Command Pattern

Vị trí áp dụng: áp dụng tại backend cho chức năng gửi thông báo (cụ thể trong `notification-service`, hỗ trợ xử lý và gửi đa dạng các loại email như: email mã OTP, email chào mừng, và email xác nhận lịch khám).

Lý do áp dụng: nhận thấy trong hệ thống có nhiều kịch bản gửi thông báo khác nhau, mỗi kịch bản lại yêu cầu trích xuất dữ liệu và sử dụng mẫu hiển thị riêng biệt. Do đó, áp dụng Command Pattern giúp đóng gói từng yêu cầu gửi thông báo thành một đối tượng lệnh độc lập, tách biệt hoàn toàn nơi tiếp nhận khỏi logic thực thi, giúp dễ dàng mở rộng thêm các loại thông báo mới sau này mà không làm ảnh hưởng code cũ, đồng thời tạo tiền đề để đưa các tác vụ gửi email vào hàng đợi và tự động thử lại khi lỗi mạng.

Sơ đồ lớp:



Hình 25. Sơ đồ lớp Command Pattern

Cách mở rộng: Khi hệ thống mở rộng cần thêm nhiều lệnh gửi thông báo ta có thể thực hiện viết các lớp hiện thực lớp NotificationCommand sau đó override hàm execute.

7.6 Adapter Pattern

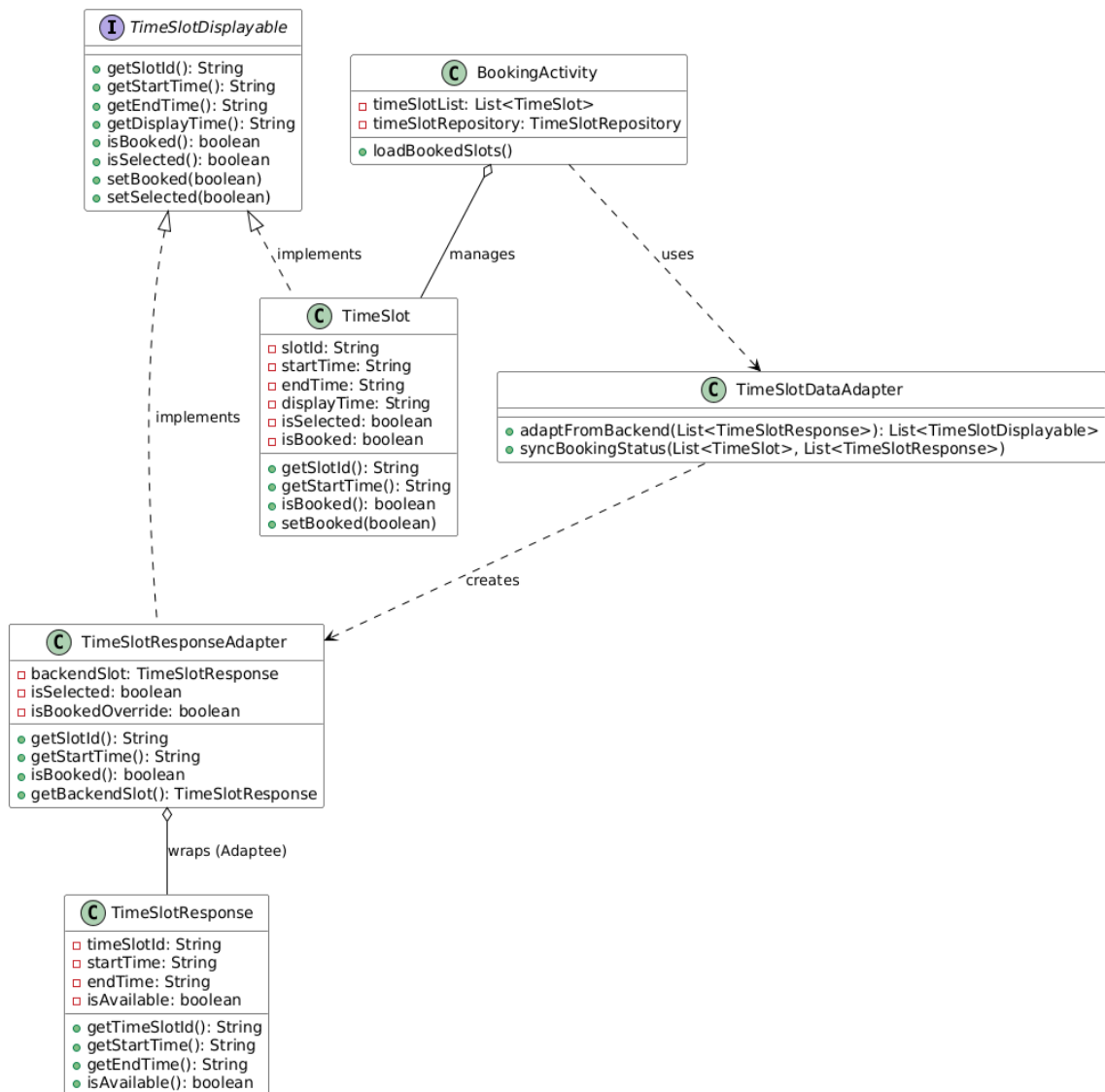
Vị trí áp dụng: Pattern được áp dụng tại tầng xử lý dữ liệu trung gian ở phía ứng dụng di động). Cụ thể, nó được dùng để chuyển đổi đối tượng dữ liệu khung giờ khám nhận được từ Backend API thành đối tượng hiển thị tương thích với giao diện người dùng trên Frontend.

Lý do áp dụng:

- Giải quyết sự bất đồng bộ về cấu trúc dữ liệu: Backend trả về chứa dữ liệu thô tối ưu cho việc lưu trữ và truyền tải qua mạng. Tuy nhiên, các thành phần giao diện của Android lại cần dữ liệu đã được định dạng chuẩn bị sẵn để hiển thị.
- Tính tách biệt: Giao diện chỉ làm việc với một Interface chuẩn do Frontend tự định nghĩa. Nếu cấu trúc API của Backend thay đổi trong tương lai, phía Frontend không cần phải đi sửa hàng loạt code ở các

màn hình UI, mà chỉ cần cập nhật lại logic chuyển đổi bên trong lớp Adapter.

Sơ đồ lớp:



Hình 26. Sơ đồ lớp Adapter Pattern

Cách mở rộng: Khả năng mở rộng của Adapter Pattern thể hiện rõ khi hệ thống cần tích hợp dữ liệu từ nhiều nguồn khác nhau. Giả sử trong tương lai, ứng dụng cho phép bác sĩ đồng bộ lịch làm việc từ hệ thống bên thứ 3 (như Google Calendar) thay

vì chỉ lấy từ Backend của bệnh viện. Dữ liệu trả về từ Google API sẽ là một lớp hoàn toàn khác.

7.7 Observer Pattern

Vị trí áp dụng: Áp dụng vào phần Đặt lịch, cho các services lắng nghe sự kiện Đặt lịch thành công/ Hủy lịch, từ đó throw ra events để các listeners catch và gọi về services để tiếp tục thực hiện luồng nghiệp vụ bằng các clients trong appointment-service.

Lý do áp dụng:

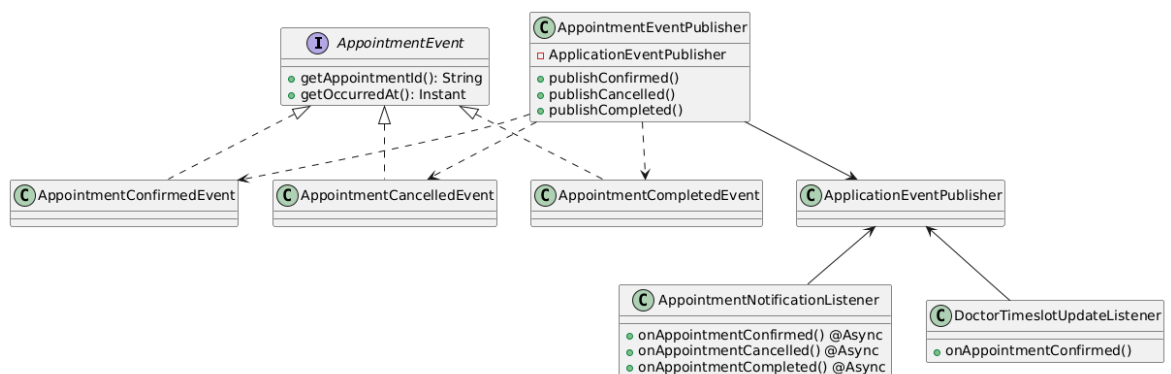
Trước khi dùng Observer (code không có event):

- Handler ngày càng phình to
- Khó test từng phần
- Khi lỗi email → ảnh hưởng cả appointment flow
- Thêm tính năng mới → sửa Handler

Sau khi dùng Observer:

- Decouple: Handler không cần biết ai lắng nghe
- Single Responsibility: Mỗi listener chỉ làm 1 việc
- Independent Scaling: Thêm listener mới không sửa code cũ
- Async Support: Email không block main flow
- Order Control: Critical operations chạy trước (@Order)

Sơ đồ lớp:



Hình 27. Sơ đồ lớp Observer Pattern

Cách mở rộng: Thêm Event Rescheduled mới, Thêm Listener cho Event gửi SMS, Cross-Service Event

7.8 Decorator Pattern

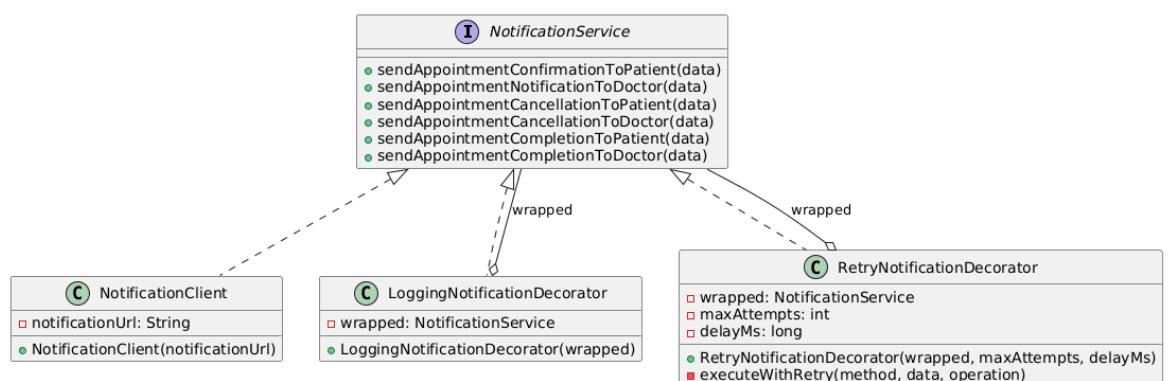
Vị trí áp dụng:

Áp dụng vào phần Client Notification trong Appointment Service

Lý do áp dụng:

- Khi gửi email đi, hệ thống cần Logging và Retry nếu fail, việc áp dụng decorator pattern sẽ giúp cho hệ thống phủ thêm các lớp Logging và Retry mà không cần phải đi sửa code quá nhiều.
- Tránh lặp code khi thêm feature mới, có thể bật tắt decorator dễ dàng bằng cách thêm feature mới vào trong service mà không cần phải sửa cả Client.
- Giảm bớt trách nhiệm cho 1 class (chứa cả 3 trách nhiệm logging, retry, gửi mail).
- Tránh vi phạm Single Responsibility Principle và Open/Closed Principle.
- Dễ dàng unit tests, phù hợp với hệ thống Microservices.

Sơ đồ lớp:



Hình 28. Sơ đồ lớp Decorator Pattern

Cách mở rộng:

- Thêm features mới vào Service và khai báo thêm vào trong chain
Chain: Retry(Logging(Metrics(Circuit(NotificationClient()))))

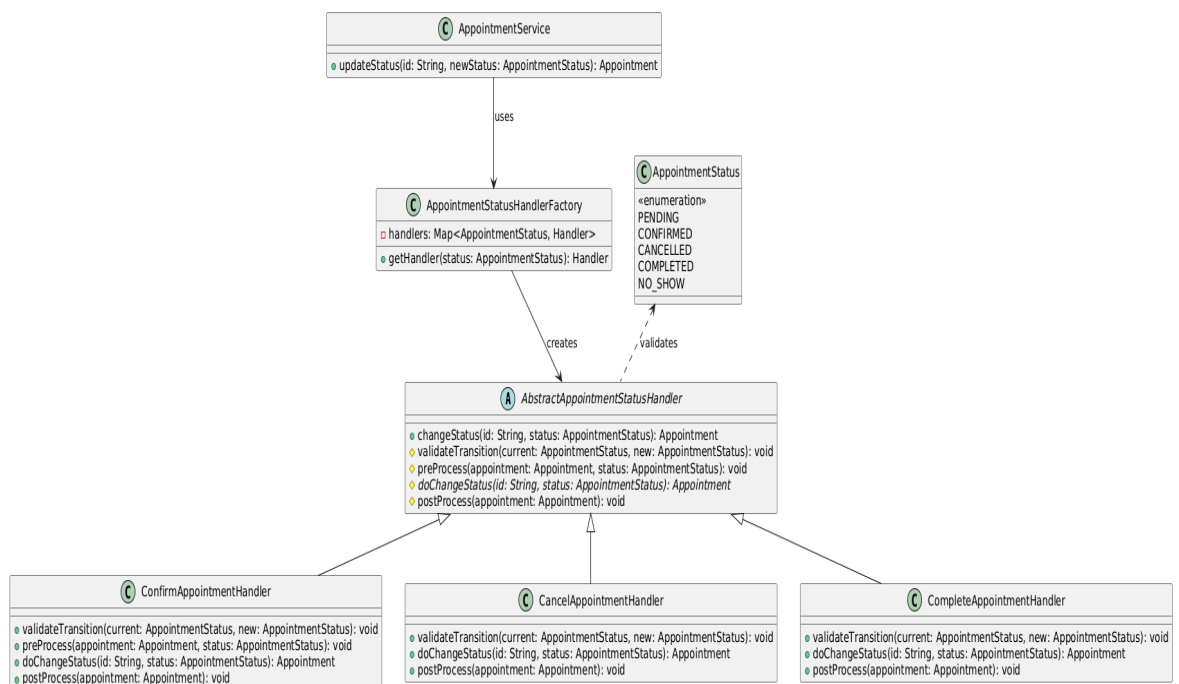
7.9 State Pattern

Vị trí áp dụng: Đưa các status của Appointment Service sang thành dạng enum nhằm chuẩn hóa dữ liệu, tránh sai status, linh hoạt giữa các business logics của các status.

Lý do áp dụng:

State Pattern giúp việc thêm status mới một cách dễ dàng, chỉ cần tạo thêm class và business logics, dễ dàng unit tests. Tránh được việc sử dụng quá nhiều if-else như cấu trúc truyền thống, không phải sửa toàn bộ code khi muốn thêm status mới, dễ bị quên validation và nghiệp vụ phụ khi tạo mới. Ở đây kết hợp thêm với Template Method và SimpleFactory

Sơ đồ lớp:



Hình 29. Sơ đồ lớp State Pattern

Cách mở rộng:

Thêm status mới – Rescheduled, thêm handler mới

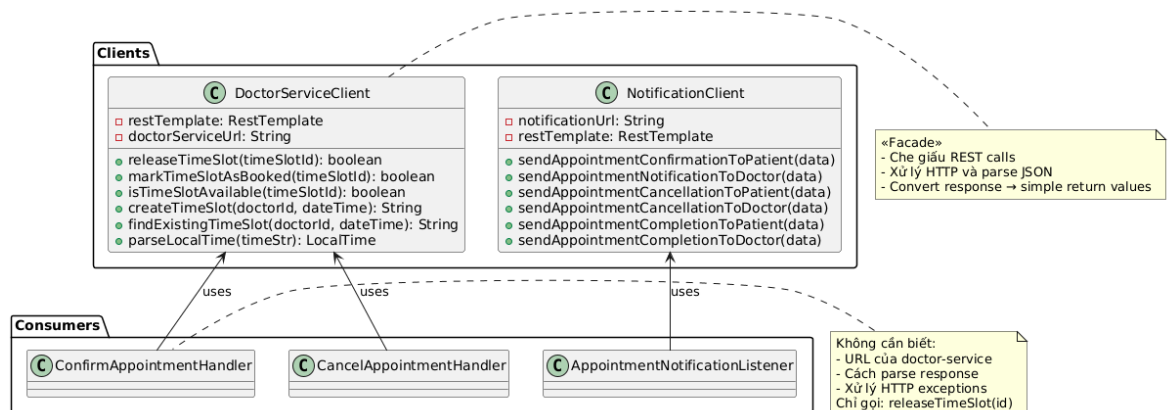
7.10 Facade Pattern

Vị trí áp dụng: DoctorServiceClient và NotificationClient - đóng vai trò facade cho các service calls

Lý do áp dụng:

- Che giấu sự phức tạp khi gọi external services
- Đơn giản hóa API cho các handler/listener
- Xử lý HTTP, parse response, handle errors tập trung 1 chỗ

Sơ đồ lớp:



Hình 30. Sơ đồ lớp Facade Pattern

Cách mở rộng:

Muốn cache → bọc gateway bằng CachedDoctorServiceGateway, đổi chiến lược cache chỉ cần swap CacheManager. Muốn chịu lỗi → bọc thêm FallbackDoctorServiceGateway. Muốn thêm behavior khác (retry, timeout...) → tạo thêm lớp Decorator mới.

7.11 Lock/Guard Pattern

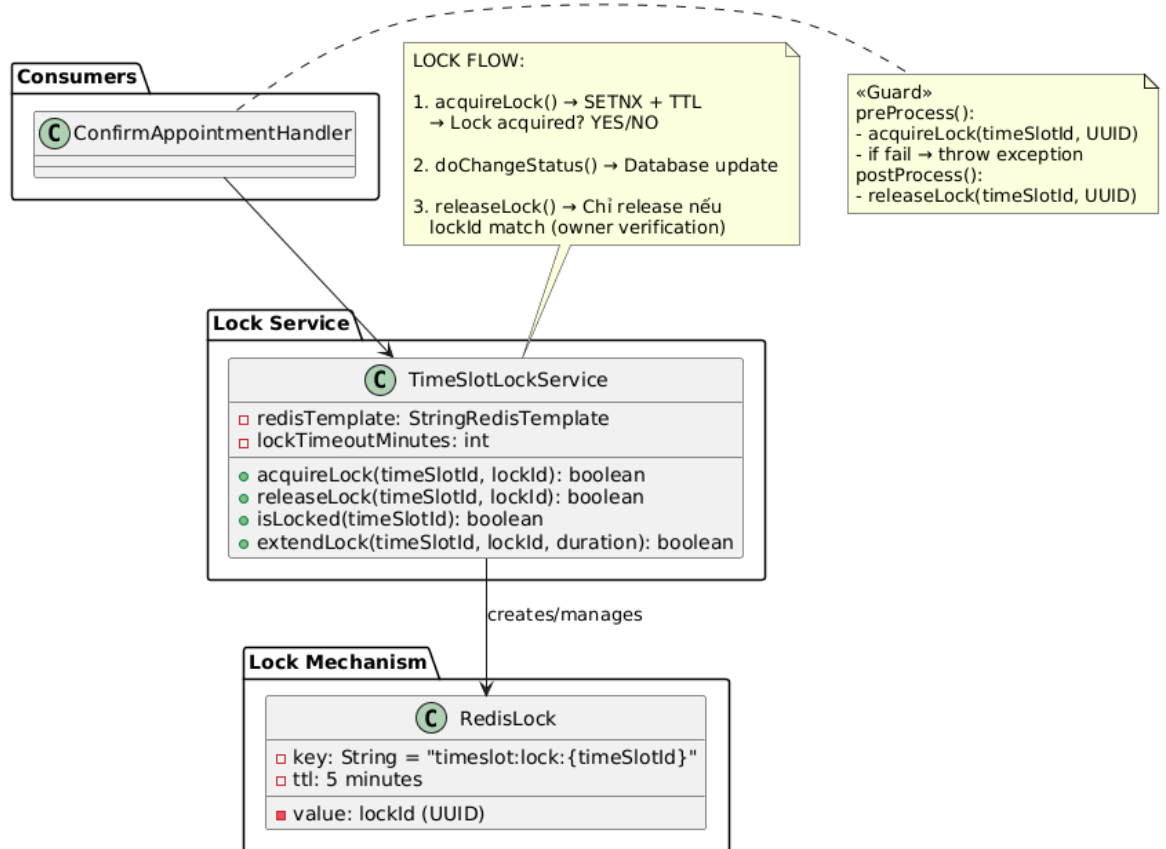
Vị trí áp dụng: TimeSlotLockService - Redis-based distributed locking

Lý do áp dụng:

- Ngăn race condition: Khi 2 người cùng book 1 timeslot cùng lúc
- Đảm bảo consistency: Chỉ 1 người giữ lock mới được confirm

- Auto-expiry: TTL ngăn deadlock khi user abandon

Sơ đồ lớp:



Hình 31. Sơ đồ lớp Lock/Guard Pattern

Cách mở rộng:

Thêm công nghệ lock mới (Zookeeper, etcd...) → implement interface DistributedLock. Cần tính năng cao cấp hơn → dùng RedissonLockAdapter. Cần thêm behavior (retry, circuit breaker...) → tạo Decorator mới bọc ngoài DistributedLock, tương tự LeasedLock.

7.12 Global Exception Handler Pattern

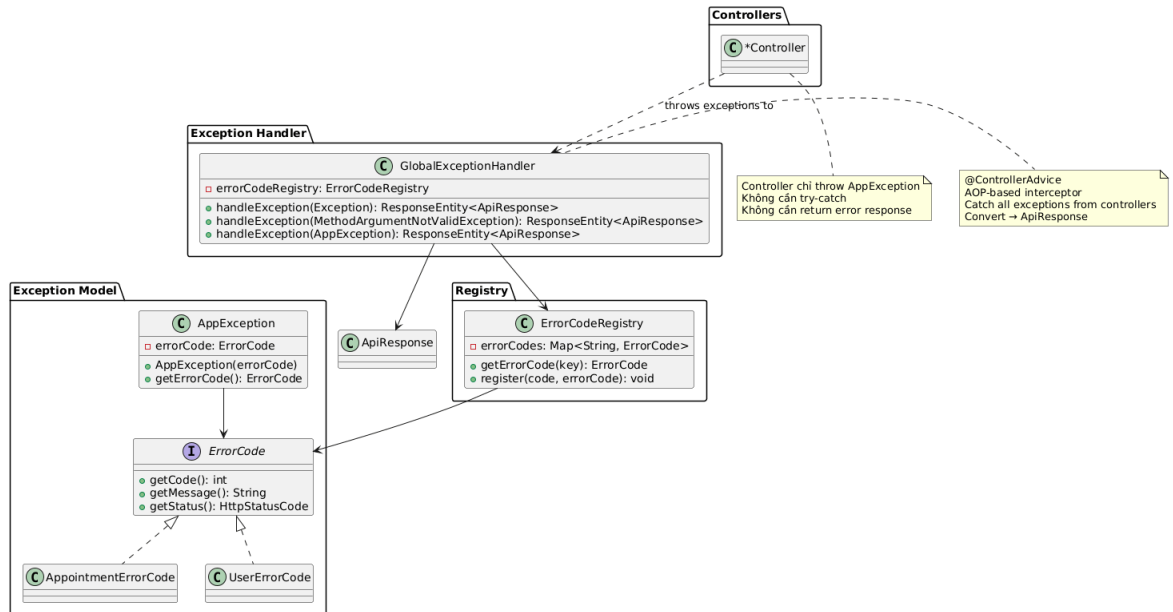
Vị trí áp dụng: GlobalExceptionHandler trong common-exception module

Lý do áp dụng:

- Centralized error handling: Tất cả exceptions xử lý 1 chỗ

- Consistent API response: Mọi API trả về format giống nhau
- Separation of concerns: Controller không lo xử lý lỗi
- Cross-cutting: Áp dụng cho tất cả controllers

Sơ đồ lớp:



Hình 32. Sơ đồ lớp Global Exception Handler Pattern

Cách mở rộng:

Thêm loại exception mới → tạo thêm Handler mới, không sửa code cũ. Thêm kênh thông báo (Slack, PagerDuty...) → thêm method vào `ExceptionLogger`. Thêm kiểu response → thêm factory method vào `ExceptionResponseBuilder`.

CHƯƠNG 8. THIẾT KẾ GIAO DIỆN

Trong hệ thống y tế thông minh (Smart Healthcare), đối tượng sử dụng hệ thống có sự phân hóa rõ rệt về nghiệp vụ, nhu cầu và quyền hạn truy cập dữ liệu. Do đó, thay vì xây dựng một ứng dụng "nguyên khối" duy nhất với các menu phân quyền phức tạp, hệ thống được định hướng thiết kế và phát triển thành hai ứng dụng di động độc lập: Ứng dụng dành cho Bệnh nhân và Ứng dụng dành cho Bác sĩ .

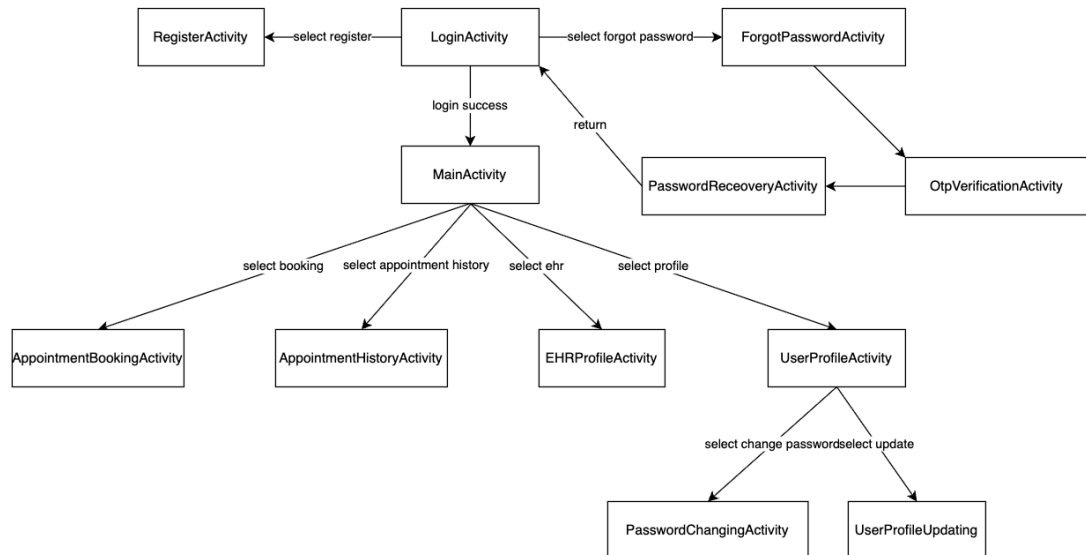
Việc phân tách hai giao diện này mang lại những ưu điểm vượt trội về mặt kiến trúc và trải nghiệm người dùng:

- Tối ưu hóa trải nghiệm đặc thù:
 - Ứng dụng Bệnh nhân được thiết kế với ngôn ngữ thiết kế thân thiện, trực quan, thao tác đơn giản (dễ dàng đặt lịch hẹn, xem hồ sơ sức khỏe, nhận thông báo).
 - Ứng dụng Bác sĩ được thiết kế như một không gian làm việc chuyên nghiệp , tối ưu hóa không gian hiển thị cho các nghiệp vụ quản lý lịch trình mật độ cao, xử lý thao tác xác nhận/hủy lịch khám nhanh chóng.
- Bảo mật và Phân quyền: Việc cô lập hai ứng dụng ở cấp độ Client giúp giảm thiểu rủi ro lộ lọt dữ liệu. Ứng dụng Bác sĩ có các luồng xác thực riêng và chỉ gọi đến các API dành riêng cho nghiệp vụ y tế, đảm bảo tuân thủ tính bảo mật của hồ sơ bệnh án.
- Hiệu năng và Tính mở rộng: Cả hai ứng dụng đều hoạt động như các Client độc lập trên nền tảng Android, giao tiếp với hệ thống Backend thông qua các RESTful API. Thiết kế này giúp kích thước ứng dụng nhẹ hơn và dễ dàng bảo trì, nâng cấp các tính năng độc lập trong tương lai mà không làm ảnh hưởng đến đối tượng còn lại.

Chi tiết thiết kế từng giao diện cho các vai trò:

8.1 Giao diện cho bệnh nhân

Tổng quát sơ đồ kiến trúc giao diện:



Hình 33. Sơ đồ tổng quát tương tác giao diện bệnh nhân

Thiết kế chi tiết cho từng màn hình:

Smart Healthcare

Tên đăng nhập

Mật khẩu

Đăng Nhập

[Đăng ký](#) | [Quên mật khẩu?](#)

Màn hình đăng nhập

Tạo Tài Khoản Y Tế

Họ và tên (*)

Số CCCD (12 số) (*)

Số điện thoại (10 số) (*)

SĐT khẩn cấp (10 số) (*)

Email

Địa chỉ

Công việc (Tùy chọn)

Số bảo hiểm y tế (Tùy chọn)

Nhóm máu (Tùy chọn)

Cân nặng (kg) (*) Chiều cao (cm) (*)

Mật khẩu (*)

Chọn ngày sinh (*)

Giới tính (*)

Màn hình đăng ký

Quên Mật Khẩu

Vui lòng nhập email đã đăng ký. Chúng tôi sẽ gửi mã OTP để đặt lại mật khẩu.

Nhập Email của bạn

Gửi Mã OTP

[Quay lại Đăng nhập](#)

Màn hình quên mật khẩu

Xác thực OTP

Mã OTP đã được gửi tới: ntu318485@gmail.com

N h ậ p m ã O T P

Thời gian còn lại: 04:29

Xác Nhận OTP

[Gửi lại mã OTP](#)

Màn hình nhậ mã OTP

SMART HEALTHCARE

Tạo mật khẩu mới

Đang đổi mật khẩu cho tài khoản: ntu318485@gmail.com

Nhập mật khẩu mới

Xác nhận lại mật khẩu

Cập nhật mật khẩu

Màn hình đặt lại mật khẩu

Xin chào, Tuấn

Chúc bạn một ngày khỏe mạnh.

Các Chức Năng Chính

- Thông Tin Cá Nhân**
Cập nhật hồ sơ, ảnh đại diện và thông tin liên lạc của bạn.
- Lịch Sử Đặt Lịch**
Xem lại tất cả các lần khám bệnh trước đây.
- Đặt Lịch Khám**
Tìm bác sĩ, chuyên khoa và chọn giờ hẹn khám bệnh.
- Hồ Sơ EHR**
Truy cập bệnh án điện tử, kết quả xét nghiệm và đơn thuốc của bạn.

© Copyright by TCT

Màn hình chính

11:28

Đặt Lịch Khám

Chọn bệnh viện, khoa và bác sĩ phù hợp

- Chọn Bệnh Viện**
Bệnh viện Từ Dũ ☒
- Chọn Khoa Khám**
Khoa Sản ☒
- Chọn Bác Sĩ**
 - Lê Quang An**
62e2321f-50fa-446b-ac71-43c55630e5c9 ▶
 - Đặng Thị An**
62e2321f-50fa-446b-ac71-43c55630e5c9 ▶
 - Võ Thị Tùng**
62e2321f-50fa-446b-ac71-43c55630e5c9 ▶

Màn hình đặt lịch

11:29

Lịch Sử Đặt Khám

08/04/2026 - 15:00 CONFIRMED

BS. Lê Quang An - Khoa Sản
Bệnh viện Từ Dũ
Lý do: Bệnh

[Hủy lịch hẹn](#)

Màn hình lịch sử đặt lịch

11:28

Hồ sơ bệnh án điện tử (EHR)

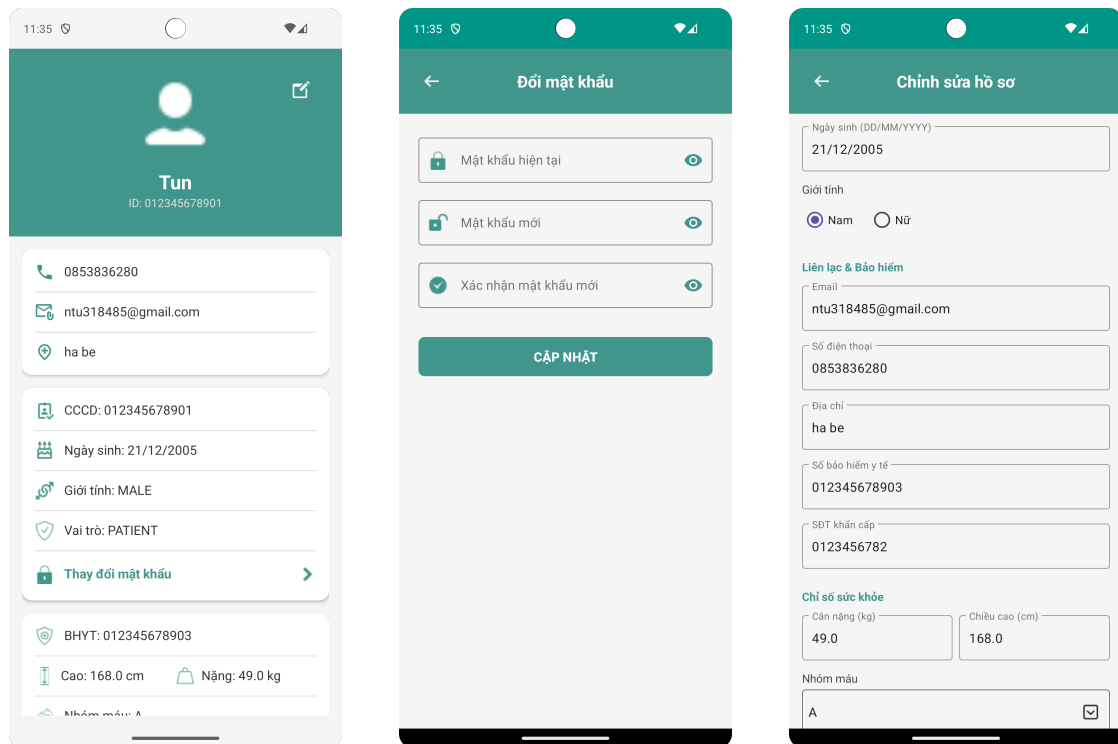
Thông tin bệnh nhân

Tun
Mã BN: a353cfed-f444-4eb6-a46c-a1f3970b12ee
Nhóm máu: A Cao: 168.0cm Nặng: 49.0kg

BHYT: 012345678903
SDT Khẩn cấp: 0123456782

Lịch sử khám & Điều trị

Màn hình thông tin EHR



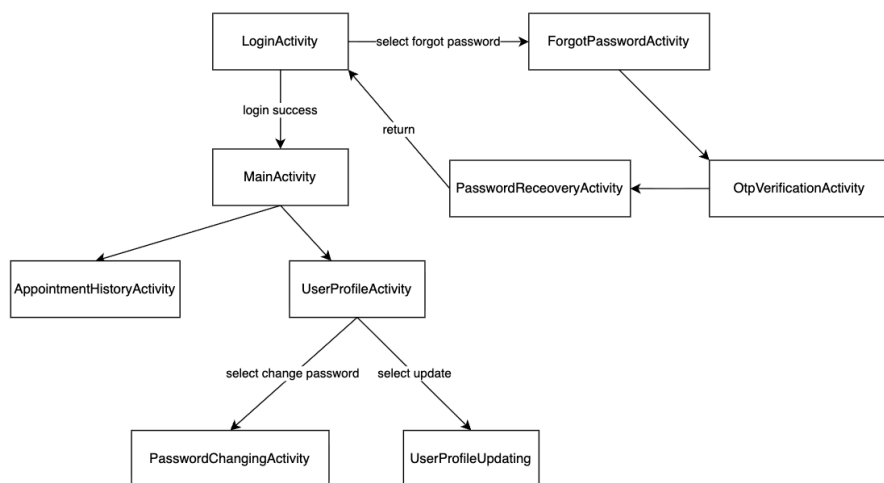
Màn hình thông tin người dùng

Màn hình thay đổi mật khẩu

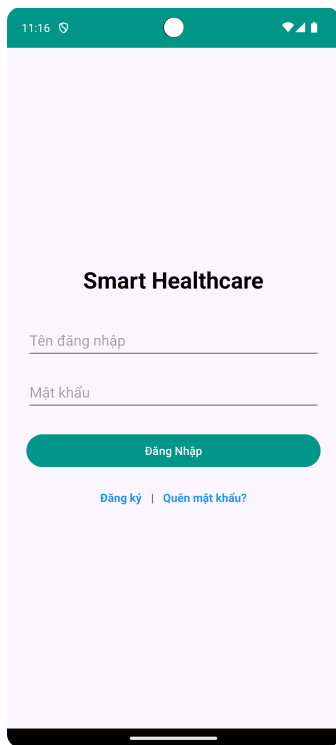
Màn hình Cập nhật thông tin

8.2 Giao diện cho bác sĩ

Tổng quát thiết kế cho từng giao diện



Hình 34. Sơ đồ tổng quát tương tác giao diện bác sĩ
Thiết kế chi tiết cho từng màn hình:



Smart Healthcare

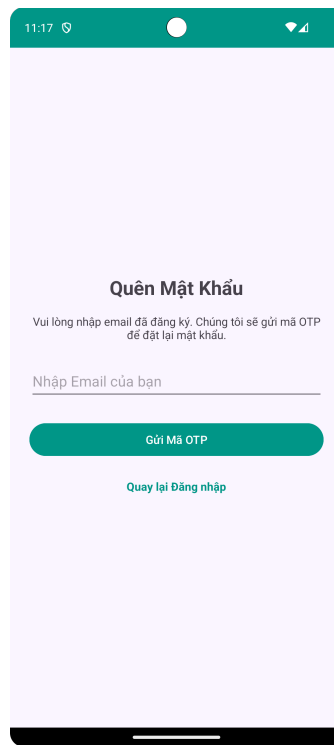
Tên đăng nhập

Mật khẩu

Đăng Nhập

[Đăng ký](#) | [Quên mật khẩu?](#)

Màn hình đăng nhập



Quên Mật Khẩu

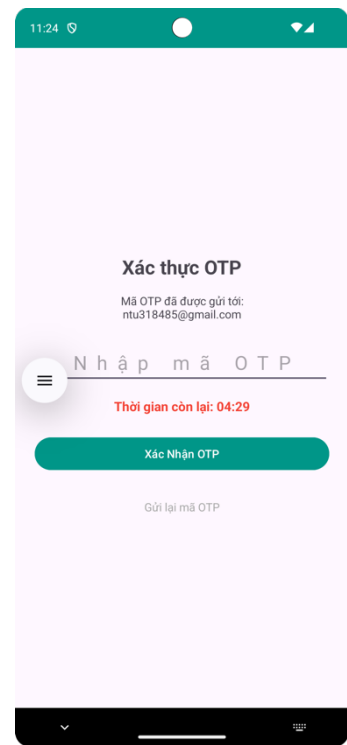
Vui lòng nhập email đã đăng ký. Chúng tôi sẽ gửi mã OTP để đặt lại mật khẩu.

Nhập Email của bạn

Gửi Mã OTP

[Quay lại Đăng nhập](#)

Màn hình quên mật khẩu



Xác thực OTP

Mã OTP đã được gửi tới: ntu318485@gmail.com

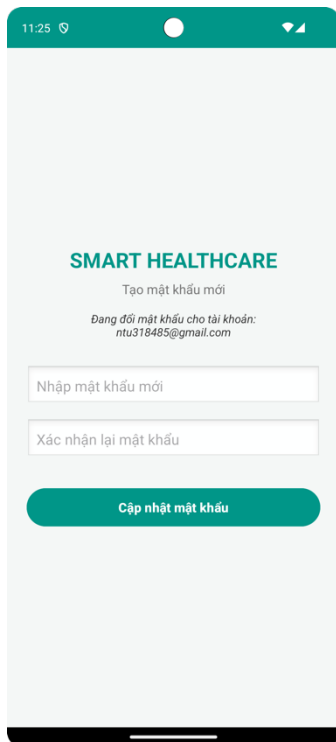
N h ậ p m ẫ O T P

Thời gian còn lại: 04:29

Xác Nhận OTP

[Gửi lại mã OTP](#)

Màn hình nhập OTP



SMART HEALTHCARE

Tạo mật khẩu mới

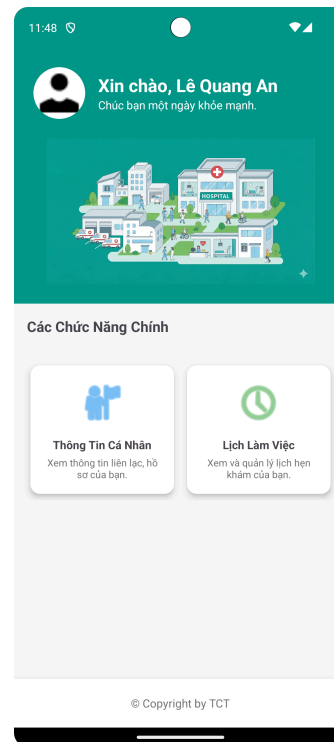
Đang đổi mật khẩu cho tài khoản: ntu318485@gmail.com

Nhập mật khẩu mới

Xác nhận lại mật khẩu

Cập nhật mật khẩu

Màn hình đặt lại mật khẩu



Xin chào, Lê Quang An

Chúc bạn một ngày khỏe mạnh.

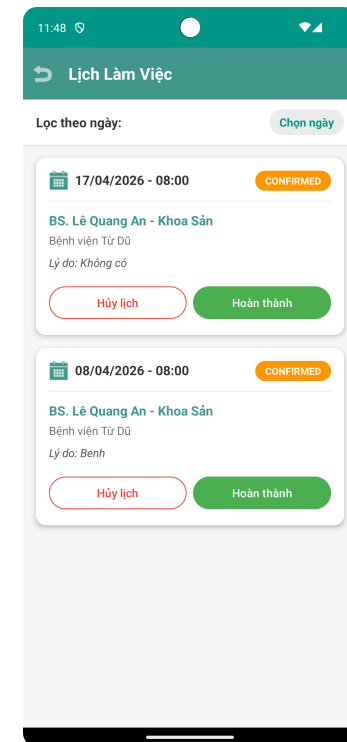
Các Chức Năng Chính

Thông Tin Cá Nhân
Xem thông tin liên lạc, hồ sơ của bạn.

Lịch Làm Việc
Xem và quản lý lịch hẹn khám của bạn.

© Copyright by TCT

Màn hình chính



Lịch Làm Việc

Lọc theo ngày: [Chọn ngày](#)

17/04/2026 - 08:00 **CONFIRMED**

BS. Lê Quang An - Khoa Sản

Bệnh viện Từ Dũ

Lý do: Không có

[Hủy lịch](#) [Hoàn thành](#)

08/04/2026 - 08:00 **CONFIRMED**

BS. Lê Quang An - Khoa Sản

Bệnh viện Từ Dũ

Lý do: Bệnh

[Hủy lịch](#) [Hoàn thành](#)

Màn hình lịch hẹn

CHƯƠNG 9. KẾT LUẬN

9.1 Kết luận

Dự án "Hệ thống Ứng dụng Y tế Thông minh (Smart Healthcare)" đã hoàn thành tốt các mục tiêu đề ra ban đầu, không chỉ ở khía cạnh đáp ứng yêu cầu nghiệp vụ thực tế mà còn ở việc tuân thủ chặt chẽ các nguyên lý kỹ thuật phần mềm. Với mục tiêu trọng tâm là môn học Mẫu thiết kế, dự án đã đạt được những kết quả nổi bật sau:

- Áp dụng thành công các Mẫu thiết kế: Hệ thống không dừng lại ở việc viết code chạy được mà hướng tới code sạch và dễ bảo trì:
 - Nhóm Creational: Ứng dụng Factory Method trong việc khởi tạo linh hoạt các thành phần giao diện cho từng trạng thái lịch hẹn khác nhau (Pending, Confirmed, Completed, Cancelled) mà không làm phình to logic của Adapter.
 - Nhóm Structural và Behavioral: Áp dụng Observer Pattern làm nền tảng cho kiến trúc MVVM ở phía Client, giúp giao diện tự động cập nhật khi dữ liệu thay đổi.
- Xây dựng Kiến trúc Microservices linh hoạt: Ở phía Server, dự án đã hiện thực hóa thành công kiến trúc phân tán với nguyên lý Database-per-service (CSDL độc lập cho từng dịch vụ như User, Doctor, Appointment). Điều này minh chứng cho khả năng thiết kế hệ thống có tính rời rạc cao.
- Tối ưu hóa trải nghiệm người dùng (UX): Việc mạnh dạn phân tách hệ thống thành hai ứng dụng di động độc lập đã giúp cá nhân hóa không gian làm việc, tăng cường bảo mật và đơn giản hóa luồng thao tác cho từng nhóm đối tượng chuyên biệt.

9.2 Hướng phát triển

Mặc dù hệ thống cơ bản đã hoàn thiện các luồng nghiệp vụ cốt lõi, nhưng tiềm năng mở rộng của dự án vẫn còn rất lớn. Trong giai đoạn tiếp theo, dự án định hướng phát triển theo các mũi nhọn sau:

9.2.1 Nâng cấp và mở rộng Mẫu thiết kế

Áp dụng Strategy Pattern cho Module Thanh toán: Khi tích hợp tính năng thanh toán viện phí, Strategy Pattern sẽ được sử dụng để hệ thống dễ dàng chuyển đổi qua lại giữa các phương thức thanh toán khác nhau (Tiền mặt, VNPay, MoMo, Thẻ tín dụng) mà không cần can thiệp sửa đổi code lõi.

Áp dụng Command Pattern cho Hồ sơ bệnh án điện tử: Nhằm tăng tính an toàn khi thao tác dữ liệu y tế, Command Pattern sẽ được tích hợp để đóng gói các thao tác nhập liệu của bác sĩ. Điều này cho phép hệ thống lưu vết toàn bộ quá trình chỉnh sửa và hỗ trợ tính năng hoàn tác (Undo/Redo) khi bác sĩ vô tình nhập sai thông tin chẩn đoán.

9.2.2 Tích hợp Trí tuệ Nhân tạo và Machine Learning

Sử dụng các mô hình học máy để phân tích sơ bộ hồ sơ sức khỏe và sinh hiệu của bệnh nhân.

Tích hợp các API Trí tuệ nhân tạo (như OpenAI hoặc Google AI) để xây dựng hệ thống Chatbot kiểm tra triệu chứng thông minh (Symptom Checker), từ đó đưa ra gợi ý giúp bệnh nhân tự động chọn đúng khoa khám và bác sĩ chuyên môn phù hợp trước khi đặt lịch.

TÀI LIỆU THAM KHẢO

- [1] E. Gamma, R. Helm, R. Johnson và J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Reading, MA: Addison-Wesley, 1994.
- [2] S. Newman, Building Microservices: Designing Fine-Grained Systems, Sebastopol, CA: O'Reilly Media, 2021.
- [3] R. S. Pressman và B. R. Maxim, Software Engineering: A Practitioner's Approach, New York, NY: McGraw-Hill Education, 2020.
- [4] R. Elmasri và S. B. Navathe, Fundamentals of Database Systems, Boston, MA: Pearson, 2015.
- [5] Spring, “Spring Boot Reference Documentation,” [Trực tuyến]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. [Đã truy cập 10 04 2026].
- [6] Google Developers, “Build your first Android app in Java,” [Trực tuyến]. Available: developer.android.com. [Đã truy cập 05 04 2026].
- [7] Square Open Source, “Retrofit - A type-safe HTTP client for Android and Java,” [Trực tuyến]. Available: <https://square.github.io/retrofit/>. [Đã truy cập 20 03 2026].
- [8] Docker Inc., “Docker Documentation,” [Trực tuyến]. Available: <https://docs.docker.com/>. [Đã truy cập 15 03 2026].